Response Document for I-Rex: An Interactive Debugger for SQL

MAIN CHANGES

We thank the reviewers for giving us the opportunity to revise the paper, and their constructive feedback that helps us significantly improve the paper. We first describe a summary of changes in the revision below and then list detailed responses to comments from each reviewer.

- We have revised Section 3 on debugging paradigm throughout (including a new Section 3.3) to explain 1) the currently supported SQL fragments, 2) the generality and limitation of our debugging paradigm, 3) how to extend I-Rex to handle additional SQL constructs, and 4) how to use data provenance information judiciously to support tracing and pinning.
- We have revised Section 4 on system and optimization (including a completely revamped Section 4.2) to clarify how different optimization techniques are combined and applied automatically, and to better explain system assumptions and workflow.
- For Section 5 on performance experiments, we have completely rerun all evaluations (including the ones reported in the appendix of the full version [36]) with the most up-to-date PostgreSQL 18. We further evaluated the end-to-end latency of each debugging operation and reported additional frontend overhead. Finally, we added a discussion of I-Rex's scalability limitations and proposed new research ideas for future work.
- For Section 6 on user study, we have included students' post-study feedback on I-Rex and discussed its implications for the main result. We also discussed the limitations of the user study and highlighted potential future improvements. Finally, we conducted an additional case study on the potential of using LLMs directly for debugging (see the full version [36] for details, which we we cannot include in the paper because of space limits) and discussed how it complements but does not replace I-Rex.
- In Section 7 on related work, we further clarified the differences between I-Rex and other systems (especially Habitat and DESQL) and the reasons why they are not suitable as candidates for direct experimental comparison.
- For all sections, we made sure that all labels for examples and theorems were corrected.

In the following, we have color-coded updates and responses specific to comments:

- Changes in response to multiple reviewers are in Purple.
- Other generic improvements are also in Purple.
- Reviewer #1 in Blue.
- Reviewer #2 in Magenta.
- Reviewer #3 in Red.

REVIEWER #1

Comment W1, D1. While the paper addresses TPC-H queries, many of the user study examples focus on relatively simple queries, which is understandable as these users are bachelor students. However, I-Rex's effectiveness for highly complex, multi-nested OLAP queries with hundreds of lines remains unclear.

The performance experiments use TPC-H, which includes complex joins and subqueries. However, the user study only utilized two problems (P1 and P2) with 2-3 bugs each in an educational context. Extending the user study with real-world OLAP queries to see if the step-into and pinning help expert SQL developers debugging complex queries will improve the paper.

Response. Thank you for pointing out this important gap. Indeed, we have designed I-Rex to handle complex and multi-nested queries, but our user study involved relatively simple queries, both because our most accessible user pool consists of college students, and because we expect low uptake/completion rate for very complex debugging tasks. That said, we believe the study with simple queries still provide useful signal on the effectiveness of I-Rex in general. To draw an analogy to general-purpose debuggers like GDB for C programs, developers do not debug a massive C program by analyzing the entire codebase at once; they focus on small parts and set breakpoints and step into isolated function frames. I-Rex applies the same paradigm to SQL. By treating each nested subquery or `WITH` clause as an independent "function call," users can drill down into specific blocks and debug them in a localized fashion.

We have revised Section 6 to discuss the limitation of the current user study and furture improvements. We have also included discussion about students' feedback. We acknowledge that debugging habits and preferences are subjective and vary from user to user. Instead of trying to be the "complete" solution to the SQL debugging problem, I-Rex provides a useful and badly needed addition to our debugging toolbox.

Comment W2, D2. The paper does not compare I-Rex's interactive approach to the increasingly common use of LLMs (e.g., ChatGPT, Claude) for SQL debugging, which is a significant modern alternative.

The authors acknowledge the study was conducted before the release of LLMs like ChatGPT. Since then, LLM-based SQL debugging has become a baseline, both in educational and industry contexts [1,2]. The paper would benefit from a discussion and experiment comparing I-Rex against LLMs.

[1] https://aclanthology.org/2025.acl-industry.71.pdf
[2] https://openreview.net/pdf?id=8Fm6OKFuRv

Response. We appreciate the reviewer raising this point. We have added a discussion in Section 6 to include the above and other relevant literatures. We also clarified that I-Rex and LLMs are orthogonal, complementary tools for debugging, rather than direct competitors:

- **Different purpose & users:** LLMs directly interpret queries to provide quick fixes, which is ideal for users who just want a patched query. In contrast, I-Rex visually reveals the step-by-step logical execution under the hood, targeting users who want to deeply understand why a query behaves the way it does, and corner cases that are difficult to foresee simply by inspecting the queries alone.
- **Accuracy & synergy:** LLMs can still make mistakes on queries (even simple ones), and their performance is variable and unreliable. We highlight this via a new small case study using the latest models (details in [36] and brief discussion in Section 6). I-Rex, on the other hand, allows us to debug queries in a deterministic, reproducible, and efficient fashion (e.g., jumping directly to points of interest). An exciting avenue of future research would be to let agentic AI to use I-Rex to improve its own iterative debugging process in terms of both accuracy and speed.

Comment W3. The system seems to lack support for recursive CTEs, window functions, and lateral joins, which are essential for many analytical workloads.

Response. We acknowledge such limitations of I-Rex. While we see no fundamental limitations in our framework that would prevent I-Rex from handling these constructs, they undoubtedly require a substantial amount of additional design and implementation work. We have added a new Section 3.3 to discuss the limitation and potential extension of I-Rex. We plan to open source I-Rex in the near future, so that it will keep evolving to support more SQL constructs and more debugging features.

Comment D3. Experiments show the impact of bloom filter and other optimizations are impressive. However, the paper should also report the end-to-end latency, per-click latency for pagination or pinning of an interactive session.

Response. We have revised Section 5 to report end-to-end latency of each debugging operation. To summarize, besides the time to compute milestones / pages, the only additional overhead comes from data rendering in the frontend and remains low thanks to our paginated design.

Comment D4. I-Rex relies on IIDs to ensure reproducible ordering. Is it possible that the underlying data changes via a concurrent update during a debugging session?

Response. I-Rex is designed under the assumption that the database instance being accessed during a debugging session remains static. We have added a footnote in Section 4.1 to clarify this assumption. Generally speaking, debugging a query while the underlying database is constantly changing may not be a good idea in the first place. Systems that support multiversion-concurrency control can still offer I-Rex a stable read-only snapshot for debugging, but to support debugging in a truly dynamic setting—assuming we agree on what debugging should mean in such a setting—I-Rex's frontend must refresh the milestones when underlying data updates are detected, and additional research on how to updates them incrementally and efficiently might be required to maintain interactivity.

REVIEWER #2

Comment W1. It isn't completely clear from the paper how general the solution is. In several cases such as whether to use a Bloom filter or how IIDs are chosen, it isn't clear whether a human is involved in applying I-Rex to a particular query / dataset.

Response. I-Rex's debugging paradigm is general and is designed to capture the logical execution flow of even complex queries, and we have added a new Section 3.3 to discuss its generality and limitations. As for the system aspect, the choice of IIDs and the application of Bloom filters, sargable filters and IID filters are all done automatically by the I-Rex backend through query rewrites, and the users are not involved in or aware of the optimization. We realize that our original submission lack a more detailed explanation of how the overall system works — hence, in this version, we have completely rewritten Section 4.2 to explain things better.

Comment W2. While the example introduced in Figure 1 helps give the intuition for the system, it would be helpful to have a clear system model and overall description of the steps by which a SQL query is rewritten for debugging, even if space only permits "highlights" to be described in detail.

Response. Thanks for the suggestion. We have completely rewritten Section 4.2 to explain things better, and move some details to the full version instead.

Comment W3. Both evaluations demonstrate some weaknesses in I-Rex. Table 2 suggests that "interactive" operations may up to a minute even with all optimizations enabled and Figures 8 and 9 suggest that I-Rex doesn't help students find any more bugs, as acknowledged in the text (even if they find them faster).

Response. Thanks for raising these points, which are crucial for future development.

Performance: We acknowledge that interactivity remains a challenging issue for very expensive queries. The bottomline is that I-Rex cannot magically solve the query optimization problem itself — if the query is fundamentally hard to optimize and expensive to execute, we cannot expect it to run faster in the debugging mode. In fact, the contrary is true in general: running a program in the debugging mode usually requires taking a performance hit. In the context of SQL (correctness) debugging, since we want to debug the canonical/logical execution (which is more meaningful than debugging an optimized execution plan that doesn't resemble the original query at all), we should expect to pay an even higher premium. Using I-Rex's novel approach, however, we can 1) try to match the performance of the optimized execution even though we are debugging the canonical execution, and 2) avoid the full price of data shipping (which can be problemetic even for optimized execution). Both of these are possible because we can "mimic" a small window of canonical execution by executing an optimized query. However, doing so requires us to execute the milestone queries, which are aggregated versions of subquries in the original query, which can still take long. Promising future work directions include faster, approximate milestone computation as well as on-demand milestone computation. We have added a discussion in Section 5.3 to acknowledge the challenge and opportunities.

Also, this specific one-minute time in the old Table 2 already came down to 37 seconds in the current Table 2 (apparently due to faster CPU rather than newer PostgreSQL version). We also note that our evaluation was conducted under a setting where parallelism was turned off (`max_parallel_workers_per_gather = 0`) to make results easier to interpret without potential confounding effects of parallelization. We tried instead setting `max_parallel_workers_per_gather = 2` (the PostgreSQL default) and restoring all other configuration parameters to their defaults as well (with much lower `work_mem = 4MB` and `shared_buffer = 128MB`). Under such a setting, the longest running query in Table 2 takes around 166s and 35s for baseline and optimized page-fetch query, respectively, compared with 146s and 37s in the current Table 2. Thus, there is evidence that debugging latency will improve generally in setups configured for performance and scalability. (Of course, the discussion above about how efficient execution/debugging remains challenging for very large databases still applies).

Effectiveness: Indeed, the current study only proves that I-Rex accelerates bug finding, but does not show that I-Rex finds new bugs. We have added a new discussion at the end of Section 6 to discuss the limitations of the user study, and what potential adjustments can be made to help demonstrate more utility of I-Rex.

Comment D1. Overall I think this is interesting work and should be of interest to anyone teaching database courses. It could have been directed to an CS Education venue rather than VLDB, so my evaluation is colored by the novelty of the work from a DBMS systems point of view.

Response. Thank you for the recognition. Besides CS education, we also hope that I-Rex can raise the awareness of the importance of query debugging and writing semantically correct queries (even in the era of LLM), and help spark more research ideas in topics of interest to core database research. For example, debugging a slow logical (canonical) execution is practically feasible only because of innovations on query processing and optimization.

Comment D2. I was confused by "Theorem 2.1" and "Theorem 2.2" – I think these should both be "Example" but there are already other examples so I'm not sure. It is also distracting to me that they appear in figures and out of order (2.2 before 2.1).

Response. We apologize for the confusion. We have made a careful pass over the paper and corrected these issues.

Comment D3. Ideally introduce the core concepts and terminology independent of the running example. For example, "input combo" could be deferred until section 3.1.

Response. Thank you for the suggestion. We introduce the terminology "input combo" early with the debugging example for the ease of illustration. We have revised the paper to introduce this terminology informally in Section 2 and formally introduce it along with all other terminologies in Section 3.

Comment D4. The evaluation seems several years old. In particular, there have been two major PostgreSQL releases since version 16 (btw what was the minor version?). I have no reason to expect that the results would be dramatically different on version 18, but is there a reason they were run on an old release?

Response. We have rerun all experiments (including those in [36]) using the most up-to-date PostgreSQL 18.3 (released on 2/26/2026) and have updated Section 5 accordingly. However, due to hardware upgrades at our institution, we had to run the new set of experiments on a server with faster CPUs (all other settings were kept exactly the same as in the previous run). As a result, the running time for all experiments is overall shorter than the previous run. However, we did not observe any notable difference in trends in the experimental results between the two PostgreSQL releases, so our conclusion about the effectiveness of the optimization techniques remains unchanged.

REVIEWER #3

Comment W1. Despite S1, there is little here around user experience. In particular, Fig 4 is the only hint that users may have to scroll through very large numbers of rows. Pagination is the technical solution, however it is not clear how the users are expected to efficiently "zoom in" to the problem symptom (as in the simple example)

Response. Thanks for raising a valid concern. Indeed, I-Rex does not by itself suggest potential points of interest. On the other hand, if the user spots a problematic final result row, pinning alows paginations of earlier stages to automatically zoom in on the source of the problem without searching or browsing. Similarly, if the user why a particular input combo doesn't contribute to the final result, forwarding tracing serves the similar purposes of zooming in on subsequent stages. That said, I-Rex still relies on the user to come up with some *initial* clue of where the problem lies. The new Section 3.3 acknowledges this limitation and suggests more powerful ways to examine content beyond pagnited browsing (e.g., searching and pinning by arbitrary filters) as future work. Ultimately, however, we do not expect I-Rex to be able to identify bugs automatically; rather, it helps the user understand buggy behaviors efficiently, and it is complementary to approaches that make suggestions (e.g., using LLMs, as discussed in our response to R1W2).

Comment W2. A sizeable collection of existing tools and systems is discussed in the related work, however none of them is used as a baseline in the experiments. Why?

Response. We apologize for the confusion. Many of the discussed tools (e.g., Cosette, SQLRepair, QED) strictly require a known, correct reference query to function. I-Rex operates in environments where no reference query is available. Such a fundamental difference makes comparison impossible.

Other tools serve a different purpose from debugging (e.g., data exploration, test data generation, checking if a query contains errors in a pre-defined list). While their approach can potentially be used for spotting errors, they are complementary to I-Rex and not directly comparable.

DESQL and Habitat are the closest to I-Rex in terms of goals, but DESQL focuses debugging queries in Spark, and there is no available public codebase for Habitat. Therefore, none can be compared

experimentally. That said, these approaches and functionalities differ from I-Rex even on a high level. For example, DESQL shows intermediate results from an operator-based execution of the query and does not support correlated subqueries or row-level tracing. Similarly, Habitat presents a set-oriented view of SQL execution, executing queries in full and shows their results in bulk, and it relies on users to formulate focus filters manually to narrow down the scope of execution. We have further revised Section 7 to clarify.

Comment W3. the user study indicates that I-Rex may be more efficient than the alternative (using any other tool or method), but not more effective (similar success rate in bug discovery). it is also a bit limited in its scope (2 queries, 2/3 bugs). Is there enough coverage for a practical user tool?

Response. Indeed, our user study is limited in scope by both time and subject pool for practical reasons; as such, it does not estalish that I-Rex helps find *more* bugs (beyond the argument that it will help acclimated users find more bugs under a time constraint). We acknowledge the limitations of our user study in new discussion added at the end of Section 6, and suggest future directions to improve it to validate the effectiveness of I-Rex.

Comment D1. Not entirely clear to me how the 3 optimisations are combined (or whether they combine at all?). Each one of them maps to an augmented query, for the purpose of efficient pagination, but these are all different augmentations. is there an algorithm behind these to decide how to augment?

Response. We apologize for the confusion. For page-fetch queries, we apply all three optimizations whenever applicable: 1) IID filters are *always* in place (because it also ensures the that reproduced content is correct and precise); 2) sargable filters are applied on top of IID filters when the original query block involves base table with indices, as their jobs are to trigger index scan to speed up query execution; 3) finally, Bloom filters are applied on top of the previous two optimizations when the original query block involves correlated subqueries, and they serve to prevent the execution of expensive correlated subqueries. We have added Section 4.1.4 to make this process more explicit. (Also note that these optimizations are not applicable to *milestone* queries as they do not restrict the viewing range.)

Comment D2. As part of the user study, has feedback been collected on the usability (as orthogonal to efficiency and effectiveness) of I-rex?

Response. Yes, we did collect free-form feedback from the user study, and we have added a summary and disussion near the end of Section 6.

Comment D3. Traditional data provenance techniques (e.g., semiring annotations) are cited, but also quickly dismissed because of scalability issues. Would there be a way to make use of data provenance in combination with the canonical execution approach of I-Rex? This could be briefly discussed, if relevant

Response. Thanks for the suggestion. It is not at all our intention to dismiss data provenance techniques. In fact, tracing and pinning utilize the same information as provenance. Our IIDs are also designed (at least in part) to capture provenance when convenient. For example, the composition of the IIDs for the *joined and filtered* result is akin to a semiring annotation. On the other hand, the IIDs for the group-by aggregation result rows do not capture full provenance because the overhead makes it not scalable; instead, we co-design the IIDs for the proceeding grouping result to ensure that paginated provenance information can be efficiently recovered. The key difference from previous work lies in what provenance information we capture, and when. Instead of doing so at the whole-query level, we compute relevance provenance information on demand to reduce overhead. Unfortunately, we do not have enough space to describe our optimization of on-demand provenance computation queries in the paper, but additional details can be found in the full version [36]. We have revised Section 7 and various places in Section 3 to clarify.

Comment D4. Oddly, there are multiple references to Theorems but it seems to me these should be Examples instead?

Response. We apologize for the confusion. We have carefully made a pass over the paper and corrected these issues.

I-Rex: An Interactive Debugger for SQL

Yihao Hu
yihao.hu@duke.edu
Duke University

Zian Chen
zian.chen@duke.edu
Duke University

Zhiming Leong*
Stripe

Sharan Sokhi*
Amazon

Zachary Zheng*
Meta

Alex Chao*
University of California at San Diego

Kristin Stephens-Martinez
ksm@cs.duke.edu
Duke University

Sudeepa Roy
sudeepa@cs.duke.edu
Duke University

Jun Yang
junyang@cs.duke.edu
Duke University

ABSTRACT

SQL is declarative in nature and rich in its features. Writing semantically correct SQL queries and finding logical bugs in SQL are not easy, even for experienced programmers, who are often used to the mindset of working with general-purpose programming languages (GPLs). While there are many GPL debuggers, SQL debugging has received much less attention. In this paper, we present I-Rex, a SQL debugger that enables users to inspect the logical execution of SQL queries visually and interactively to identify and potentially fix logical bugs in the queries. I-Rex draws analogies to the debugging paradigm of GPLs (e.g., stepping, watchpoints, etc.), making it easier for programmers to adopt. However, unlike debugging GPLs, which involves executing the underlying program in full to the point of interest, I-Rex allows users to jump to arbitrary points of interest by leveraging the power of the database systems, through selective materialization and query rewrites. To simplify deployment, I-Rex acts as a lightweight middleware on top of the database system; it imposes no overhead to prepare a database for debugging and maintains no state in the database systems during debugging sessions. We demonstrate the effectiveness of I-Rex through performance experiments as well as a user study in an educational setting.

1 INTRODUCTION

Relational databases form the backbone of many data-intensive applications and scalable data analytics. Despite its age, SQL continues to retain its prevalence and importance due to its highly *declarative* nature (i.e., specifying what the answer should be rather than how to compute it) and its extensive set of features that have only grown over time. However, SQL is difficult to understand and debug. In debugging general-purpose programming languages (GPLs, e.g., C++ or Python) that are typically *procedural* (i.e., explicitly describing the steps required to compute the answer), it is natural to trace the execution of programs to debug them. However, this method becomes much trickier for SQL.

As a first attempt, one may consider "tracing" a query's logical or physical plan, a tree whose leaves represent base tables and internal nodes represent relational operators. Through this approach, the user can examine the intermediate results produced by each of the plan nodes. Unfortunately, there are several problems with this approach. Firstly, the database optimizer often compiles a SQL query into a plan that bears no resemblance to the original query, making plan tracing unhelpful in finding and fixing logical bugs in the original query. Secondly, debugging is usually an iterative process: the user may examine the execution multiple times, sometimes with minor modifications to the query. However, even small changes can lead the optimizer to choose a very different plan; for example, adding or removing even a simple condition in `WHERE` can enable or disable an index-scan opportunity. Even if the physical plan remains the same, there is no guarantee that the execution and result order are reproducible. For example, the size of the buffer memory, the choice of the hash function, and variations in the speed of parallel threads at run-time can change the ordering of intermediate result rows. Such non-repeatable and seemingly inconsistent behaviors significantly complicate debugging.

Perhaps, one possible workaround would be to restrict the database optimizer to avoid optimization across syntactic blocks of a query, such that each subquery corresponds to some subtree in the plan, and the user can at least inspect the result of each subquery. However, this may result in inefficient handling of complex queries. Furthermore, correlated subqueries, a frequently used SQL construct, render this workaround ineffective for many queries. In Section 2, we give concrete examples that illustrate this challenge and demonstrate how I-Rex helps debug these types of queries.

In this work with I-Rex, we focus on finding *logical errors* instead of fixing *performance issues*, and we aim to build an interactive SQL debugger with the following desiderata:

(1) The debugger should conceptually execute a SQL query in a completely reproducible manner that is faithful to how it is written and must be easy for programmers to understand.
(2) The debugger should offer features analogous to those in GPL debuggers so that they are easy to learn and adopt.
(3) Unlike GPL debuggers, which must execute underlying programs in full to reach points of interest, this debugger should support efficient implementation of powerful features that allow the user to jump directly to points of interest.
(4) The debugger must scale to large databases and gracefully handle prohibitively large intermediate results.
(5) The debugger should be simple to run (e.g., from a remote browser) and easy to deploy on top of a database, without modifying database system internals or requiring extensive preparation of the database for debugging.

At first glance, (1) necessitates executing the SQL query "literally" using a completely unoptimized plan, which runs counter to (4). Our key insight is that, at any given point in time, the user can examine only a small "window" of the entire execution. It suffices to support fast access to any given "window" without incurring the full cost of the unoptimized plan. Supporting such accesses, along

*Work done during undergraduate study at Duke.

with (2) and (3) while respecting (5), requires novel optimization — SQL's built-in `OFFSET` and `LIMIT` constructs fail to deliver acceptable performance for interactive debugging.

To this end, I-Rex makes the following contributions:

- I-Rex introduces a novel debugging paradigm for SQL that draws many parallels to GPL debugging. Each query block is viewed as a function, and correlated subqueries are considered functions with arguments. We define the *canonical execution* of a SQL query, which is reproducible and faithful to query syntax.
- I-Rex supports various debugging features analogous to GPL debugging, including stepping through execution, pausing to examine a particular point during execution (breakpoints), pausing automatically at points of interest (watchpoints), drilling down into subqueries (stepping "into" a function), and row-level tracing (information flow analysis) in both forward (from input to output) and backward (from output to input) directions.
- While performing the canonical execution would have been extremely inefficient, we designed query optimization techniques for I-Rex to support fast "teleporting" from one point of execution to another without paying the cost of execution in between.
- I-Rex has a web-based frontend and a middleware backend that runs on top of a database system, making it easy to adopt and deploy. It imposes no overhead to prepare a database for debugging and maintains no state in the database during active debugging.
- Our performance evaluation using the TPC-H benchmark shows the scalability of I-Rex on large databases. It demonstrates the advantage of our optimization techniques over standard database (PostgreSQL) support for retrieving windows of query results.
- We evaluate the efficacy of I-Rex with a user study of 100+ students in a database course. Its findings indicate that I-Rex significantly improves students' efficiency in debugging SQL queries.

2 EXAMPLE USE OF I-REX FOR DEBUGGING

Since I-Rex conceptually executes SQL queries as they are written, it is particularly suitable for novices who are learning how SQL queries work on the logical level. It also serves as a powerful tool for novices and data professionals alike to find and fix logical bugs. This section provides a walk-through of how to use I-Rex to debug a query; we introduce the interface, concepts, and features of I-Rex.

Example 1. *Consider the toy database in Figure 1, which stores information about beers, bars serving them, and drinkers who like beers and frequent bars. We want to write a query for the following task: every time a drinker frequents a bar, they buy one bottle of every beer they like or any beer priced $2 or lower; find the expected weekly revenue of each bar and rank them by revenue from high to low. A user may come up with the following (incorrect) query:*

```
SELECT s.bar, SUM(f.times_a_week * s.price) AS revenue -- Q
FROM Serves s, Frequents f
WHERE f.bar = s.bar
AND (s.price <= 2 OR EXISTS (
        SELECT * FROM Likes l WHERE f.drinker = l.drinker -- Q_inner
    ))
GROUP BY s.bar;
```

The above query intends to first find drinkers and beers available for purchase using a join between `Serves` *and* `Frequents`*. It additionally applies the two (alternative) conditions for purchase: 1) the price is lower than $2, and 2) the beer is liked by the drinker. Then, the query*

bar	beer	price
Apex	Corona	1
Apex	Dixie	2
Edge	Amstel	4
Edge	Corona	1.5
Tavern	Amstel	3
Tavern	Erdinger	1

(a) `Serves`

drinker	bar	times
Amy	Apex	1
Ben	Edge	4
Coy	Tavern	2
Dan	Edge	3

(b) `Frequents`

drinker	beer
Amy	Erdinger
Ben	Budweiser
Ben	Dixie
Coy	Amstel
Dan	Amstel
Dan	Corona

(c) `Likes`

Figure 1: A toy database about beers, bars, and drinkers.

groups the intermediate results by bar and calculates the sum of revenue. There is a bug in the `EXISTS` *subquery Q_{inner}, but the question for now is: how would a user examine the result of Q_{inner}?*

Note that Q_{inner} is correlated*, with the value for* `f.drinker` *coming from the outer (i.e. enclosing) block. As a result, there is no way to inspect this result independently. This situation cannot be handled by a query plan with relational operators, where the result of each subtree depends on this subtree alone.*

Indeed, most database optimizers will rewrite the above query for execution such that the subquery is decorrelated. The decorrelated subquery would be a join involving both `Likes` *and* `Frequents` *to compute the original subquery for all possible* `drinker` *values in a single effort. Then, the result will be combined with the rest of the outer query. The new plan now consists of only relational operators and can be computed/debugged in a bottom-up fashion, but unfortunately, it is vastly different from the original query. Users without in-depth knowledge of query optimization will likely be confused.*

Example 2. *This incorrect query described above returns the result shown in the output table in Figure 2 for the database in Figure 1. Based on the user's knowledge of the database instance, the revenue of bar* `Edge` *seems to be higher than expected. We now walk through how to use I-Rex to debug the query, starting with this observation.*

I-Rex presents a panel of UI debugging elements for each block of the query. Figure 2 illustrates the UI for the outer query block Q (for simplicity, we do not show the actual interface here as it contains other details that may be distracting for this discussion). I-Rex shows the execution of this block in stages, from top to bottom. At the very top, I-Rex shows all input tables in `FROM`*. Note that one row from each input table is highlighted; this combination of input tuples (or "input combo," formally defined in Section 3.1) intuitively defines the current point of execution being examined. Then, I-Rex presents the "joined & filtered" result, which is the intermediate output after the* `WHERE` *clause is applied. The intermediate result row produced by the current input combo is automatically highlighted. Between this result table and the input tables, a "filter expression" tree shows how the* `WHERE` *condition evaluates over the current input combo. The user can examine the value of each subexpression and see how the truth values (color-coded) are combined by logical connectives. Following the joined & filtered result, I-Rex shows the* `GROUP BY` *result. For each group, in addition to the* `GROUP BY` *value, each group member's contribution to the final* `SUM` *aggregate is also shown. Again, the group and the member that the current input combo contributes to are automatically highlighted. Finally, the output table shows the final result of the query block.*

For the convenience of subsequent discussion, we show a symbolic identifier (e.g., s_0, f_2, j_6...) for each row. We do assign internal row identifiers, whose purposes will be explained later in Section 3, but they are not explicitly displayed by the UI.

Given the unexpectedly high revenue of `Edge`*, the user naturally wants to examine how that output row was computed. I-Rex supports*

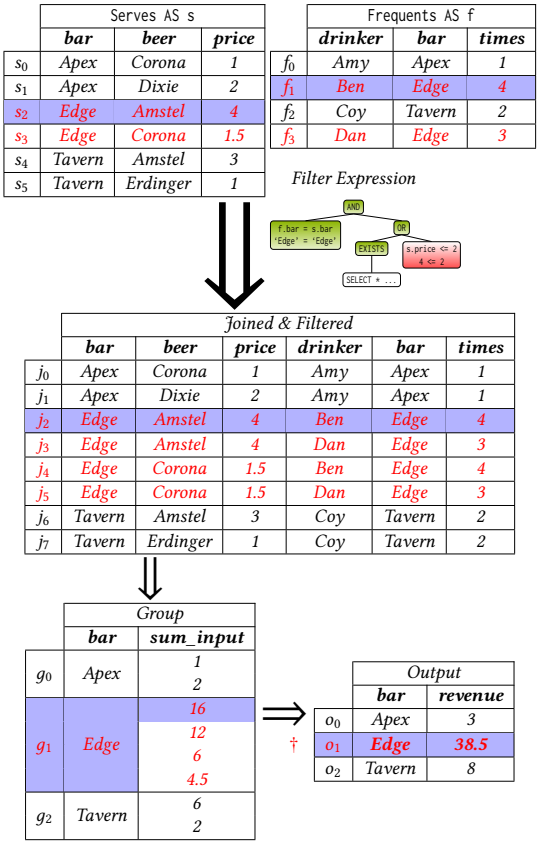


Figure 2: Debugging context for outer query block, Example 2.

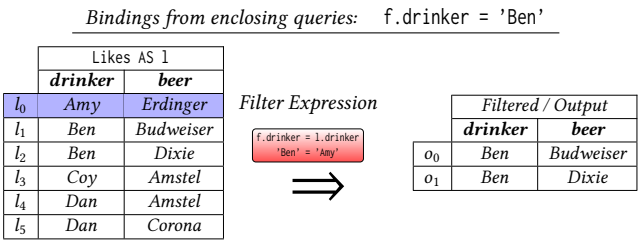


Figure 3: Debugging context for inner query block, Example 1.

tracing backward from output to input using a more general mechanism called pinning, *denoted here by the red † next to* $o_1 : \langle \texttt{Edge}, 38.5 \rangle$. *A pinned output row intuitively narrows the execution down to only parts that are "relevant" to it (which we define formally later in Section 3). As shown in Figure 2, relevant rows in upstream tables are automatically colored red. Specifically, input rows* s_2 *and* s_3 *join with* f_1 *and* f_3 *in a nested-loop fashion to produce rows* j_2 *through* j_5 *in the joined & filtered table; they are further grouped into* g_1 *in the group table before finally producing* o_1.

As soon as the user pins o_1, *I-Rex identifies the relevant input combos and "positions" execution at the first input combo in lexicographical order — this is how the input combo* $\langle s_2, f_1 \rangle$ *was activated in the first place in Figure 2. Starting with this input combo, the user can step through other input combos relevant to* o_1, *or manually choose any combo to investigate. Throughout the process, I-Rex automatically updates highlighting in downstream tables as well as the state of any expression evaluation trees, in effect supporting forward tracing.*

When examining the execution for $\langle s_2, f_1 \rangle$ *as shown in Example 2, the user notices that* $\langle s_2, f_1 \rangle$ *contributes a value of 16 to the final sum. According to the filter expression tree, the price of Amstel is higher than \$2, but* `EXISTS(`Q_{inner}`)` *returns true, meaning that Ben should like Amstel per query intention. I-Rex allows the user to "drill down" into the execution of* Q_{inner} *in this context. Recall that* Q_{inner} *is a correlated subquery. To a programmer, the analogy of evaluating* Q_{inner} *is a function call with a parameter setting for* `f.drinker`, *which takes its value from the current input combo. Hence, "drilling down" naturally corresponds to "stepping into" a function call in GPL debugging.*

Once the user drills down to Q_{inner}, *I-Rex creates a new panel for debugging this subquery block, illustrated in Figure 3. The UI makes it clear that we are executing*

```
SELECT * FROM Likes l WHERE f.drinker = l.drinker;
```

with parameter `f.drinker` *set to Ben. In this case, a quick glance at the output table or the filter expression tree for this block should reveal the problem — nothing supports that Ben likes Amstel. In fact, this subquery does not even look for Amstel. Therefore, to fix the query, we should let the outer query "call"* Q_{inner} *with an additional parameter* `s.beer`, *and let* Q_{inner} *additionally test whether* `s.beer = l.beer`.

The user can modify the original query accordingly and restart the debugging session to verify that it fixes the problem. Because the new query is syntactically similar to the old, I-Rex will produce a very consistent experience for the user: the execution of the new query will be nearly identical to the old, with the exact same stages and exact same ordering of input combos and intermediate result rows.

A Note on Scalability. While Example 2 simplifies our discussion by assuming a small database instance, realistic databases are much larger. Even with moderately sized databases, joins can easily produce large intermediate results. As we will see in Section 5, for some TPC-H benchmark queries, even with a moderate scaling factor of 1, a single intermediate result table can easily take an hour to print out entirely on the database server console. Hence, it is impractical to cache entire results at any location (database server, middleware, or user browser) or send them over the network.

Meanwhile, users generally cannot examine many rows simultaneously. Therefore, I-Rex UI supports a pagination mechanism to display each table. The user only sees a page's worth of data at once in each table. Data outside the visible page can be computed and fetched on demand (and subsequently cached or evicted). As the user interacts with the UI, I-Rex adjusts the visible portions of all displayed tables accordingly, so that each reflects the execution point defined by the current input combo. Similarly, the user can visit any page of input tables to select a new input combo for tracing; I-Rex would automatically adjust downstream table displays to show corresponding intermediate result rows.

Hence, efficient pagination is key to scalable SQL debugging. With efficient pagination, I-Rex effectively allows the user to "teleport" across points of interest without incurring the execution cost in between, making I-Rex much more powerful than GPL debugging. Pagination also provides a more manageable and less overwhelming experience for users. While most database systems support efficient pagination of input tables, it is challenging to paginate intermediate results and associated debugging information. Furthermore, I-Rex's requirement of making execution reproducible imposes specific ordering of intermediate results that complicates optimization. We discuss our solution in Section 4.

3 DEBUGGING PARADIGM

This section describes the debugging paradigm of I-Rex. We start with the data and execution model (Section 3.1), discuss various debugging operations (Section 3.2), and conclude with a summary of what SQL constructs I-Rex currently supports and how it can be further extended (Section 3.3).

3.1 Data and Execution Model

3.1.1 Table Model and IIDs. I-Rex allows users to interact with two types of tables: *base* tables and *derived* tables. Base tables are those that exist in the database, while derived tables are computed from the base tables during execution. To support reproducible execution (including ordering of intermediate result rows), we depart from the default unordered multiset semantics of SQL and instead model each table as an ordered list of rows, each associated with an *internal row id* (*IID*). A table's IIDs must be drawn from a totally ordered domain and uniquely identify the rows within the table. The IIDs do not influence the semantics of SQL query operators and should not be considered as extra columns by these operators. For example, if two rows have identical values for all columns (ignoring IID), they are still considered duplicates by SQL though their IIDs differ.

For a base table with a primary key declaration, we simply define IID to be its primary key value. Otherwise, we choose a compact `UNIQUE` key if one is available. In case that the table has no key, we use the database's internal row id (e.g., PostgreSQL's `ctid`); such ids are unique among duplicate rows. For a derived table, we define its IID according to how the table is computed. For each SQL query operator, we define a *canonical execution procedure* and a *result IID synthesis function* (more details will be provided shortly). The IID synthesis function can compute the IID for each result row on the fly during canonical execution, such that the result rows are always produced in the IID order. In addition to maintaining a reproducible order, I-Rex uses IIDs to support a variety of debugging features. Therefore, good IID designs positively impact performance, as we shall see in Section 4. Although it is technically possible to make the IID of a row simply its sequence number in the result set, such numbers are not useful by themselves for tracing, pinning, or possible query rewrite optimizations. As we will see below, most of the IIDs in I-Rex are "logical" instead of physical, encoding *data provenance* [42] helpful to tracing and pinning.

3.1.2 Query Blocks and Debugging Contexts. Given a query, I-Rex defines a *canonical execution procedure* that is always (conceptually) followed when debugging. A complex query can be viewed as a collection of syntactic blocks with dependencies among them. I-Rex models its canonical execution in terms of function calls:

- The outermost query block defines a function that computes the query result when executed.
- A subquery block defines a function that can be called by the function corresponding to its enclosing query block. A correlated subquery is analogous to a helper function with parameters, while a non-correlated subquery is comparable to a helper function without parameters. For instance, for Q_{inner} in Example 1, the caller is responsible for passing the values of external column references (e.g., `f.drinker = 'Ben'` in Figure 3) as parameters.

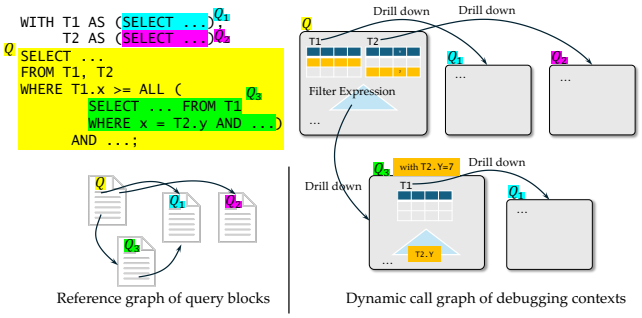


Figure 4: Static reference graph of query blocks vs. dynamic call graph of debugging contexts, for a more complex example. Note that there are two debugging contexts for Q_1 with different states; the parent of Q_3's debugging context provides the binding for `T2.Y`.

- Each table defined by `WITH` is a function that computes the table contents when the table is referenced.

Overall, the canonical execution starts by executing the function defined by the outermost query block, which then calls functions corresponding to its subqueries or tables defined by `WITH`, which may further call functions for their subqueries, etc.

As a function may be called multiple times, for each *invocation* of a function (i.e., an execution of a query block), I-Rex creates a *debugging context* as needed, analogous to an "activation frame" in GPLs. As an example, Q in Example 1 calls Q_{inner} multiple times, with potentially different `f.drinker` values as arguments, resulting in different executions. Each debugging context holds information specific to the particular execution of the query block, such as the values of external column references (i.e., parameter values). A more complex example is shown in Figure 4.

3.1.3 Canonical Execution for Each Query Block. Having discussed the overall canonical execution procedure, we now zoom in on the canonical execution of each query block. For a complex construct such as `SELECT`, we further decompose its execution into *stages*, where each stage can be seen as an operator with its own canonical execution procedure and result synthesis function. We only discuss `SELECT` with inner cross joins below; the canonical execution of SQL set/bag operations and general join expressions (including outer joins) are discussed in [36].

The first stage in a `SELECT` block is *join & filter*. Its canonical execution is nested for-loops iterating through rows, one for each input table in `FROM`, in order. In the innermost loop, we test the `WHERE` condition (which may involve calling subqueries with parameter values obtained from the loop variables). The result row IID is synthesized as a vector whose components are the IIDs of the joining input rows. Note that the lexicographic order of these vector IIDs is consistent with the row production order. Considering Example 2 and Figure 2, the IID for `Serves` is its primary key (`bar`, `beer`), and the IID for `Frequents` is its primary key (`drinker`, `bar`). Therefore, the IID for the derived joined & filtered table has the format ((`s.bar`, `s.beer`), (`f.drinker`, `f.bar`)). In particular, j_2's IID would be (s_2, f_1), where we abuse notation slightly and use s_2 and f_1 to denote their IIDs (`Edge`, `Amstel`) and (`Ben`, `Edge`) respectively.

If the block contains grouping or aggregation, a *grouping* stage will be next. Its canonical execution is a stable sort of input rows

according to the GROUP BY expressions[1] in order. Each result row starts with the GROUP BY values as columns, followed by additional columns needed to evaluate the remainder of the query (e.g. HAVING or SELECT expressions). The result row IID is synthesized as a vector whose components are the GROUP BY expressions in order, followed by the input row IID. This order puts member rows of a group together, allowing the UI to detect and display group boundaries (Figure 2). The *stable* sort ensures that the IID is consistent with the row production order. For instance, in Figure 2, the first member row of g_1 has IID (Edge, j_2), where Edge also identifies the g_1 group.

The final stage produces the *final output* for the entire SELECT block. The canonical execution processes the input rows in order. If HAVING is present, we filter out input rows whose group does not pass the HAVING condition. Next, if the previous stage is grouping, we produce one result row for each group, using the leading portion of the IID corresponding to GROUP BY expressions as the result IID. For instance, o_1 in Figure 2 would be Edge, same as g_1's. Otherwise, no aggregation is involved, and we simply produce one result row for each input row, using the same IID for the result. In either case, the IID is consistent with the result row production order.

Several special cases associated with the final stage are worth noting, including support for DISTINCT and ORDER BY; we refer interested readers to the full version [36].

3.2 Debugging Operations

We describe what operations a user can perform in a debugging context, focusing on those requiring formalization and in-depth discussion; others with straightforward semantics (e.g., visualization of expression tree) are omitted. We describe the operations mostly in the context of SELECT blocks with inner cross joins, although we have generalized them to other SQL constructs.

3.2.1 Input Combo Space and Execution Positioning. Recall that a debugging context refers to a specific execution of a query block. To represent the entire execution of a debugging context, we define its *input combo space* as an ordered set of combinations of input tuples called "*input combos*" (illustrated in Section 2), each representing a particular point in execution. The (conceptual) current point of execution is called the *active input combo* for the debugging context. For a SELECT debugging context with n input tables, the active input combo is the n input rows being examined inside the innermost loop by the canonical execution of the join & filter stage, and it is represented by an n-dimensional vector whose components are the IIDs of these input rows. For instance, the active input combo of the SELECT debugging context shown in Figure 2 is $\langle s_2, f_1 \rangle$, drawn from the input combo space $\{s_0, \ldots, s_5\} \times \{f_0, \ldots, f_3\}$.

The user can position the execution of a debugging context at a particular point using either *stepping* or *teleporting*. With *stepping*, I-Rex automatically advances the active input combo to its successor (or predecessor if stepping in reverse order) in the input combo space. For example, (ignore the pin and) suppose the active input combo in Figure 2 were $\langle s_0, f_3 \rangle$; stepping would advance it to $\langle s_1, f_0 \rangle$, followed by $\langle s_1, f_1 \rangle$, consistent with the processing order of the canonical execution. With *teleporting*, given any input table, through the paginated display, the user can jump or scroll to any page and select a particular row as active; I-Rex will then update the active input combo accordingly. Even more advanced execution positioning can be achieved by pinning, as described later.

3.2.2 Forward Tracing. An active input combo in the debugging context can produce *derivative* rows in the downstream result tables produced by the stages. Intuitively, the input combo contributes to the computation of derivative rows; formally, the input combo participates in the *how-provenance* [42] of the derivative row. As discussed in Section 2, once the active input combo is set, I-Rex automatically refreshes all visualizations of expression trees, such that they reflect evaluation over the active input combo or its derivative rows. I-Rex also automatically refreshes all result table displays, such that they show the pages containing and highlighting the derivative rows. This *forward tracing* feature allows the user to examine the effect of input rows on subsequent processing and potentially understand why a desired effect is not achieved. Note that for a SELECT block, a given input combo can contribute to at most one result row per stage, so there is no ambiguity in which derivative rows to show downstream. For example, in Figure 2, the active input combo $\langle s_2, f_1 \rangle$ has one derivative row per result table, namely j_2, the first row in g_1, and o_1. In Figure 3, there is no derivative row for the active input combo $\langle l_0 \rangle$. By design, I-Rex ensures this rule of at most one derivative per stage for other blocks (such as set/bag operations) as well.

Note that by design, across stages in a debugging context, the result IIDs naturally encode the provenance information linking rows from one stage to the next. For example, as discussed in Section 3.1.3, the IID of the joined & filtered table, synthesized from the IIDs of the input tables, essentially encodes how-provenance.

3.2.3 Pinning: Tracing and Watchpointing. We first describe the semantics of *pinning* and then discuss its use for tracing and watchpointing. Consider all tables displayed for a debugging context. I-Rex allows the user to *pin* up to one row from each of these tables. Formally, each pinned row defines a subset of the input combo space, called the row's *pinned (input combo) space*. Overall, the pinned space for the debugging context is its input combo space *intersected* with each of the pinned spaces defined by the pinned rows. Intuitively, the pinned space allows the user to narrow the execution down to the points of interest while debugging.

Consider a SELECT block joining n tables in FROM with input combo space $\mathbf{R} = \prod_{i=1}^{n} R_i$, where each R_i denotes the ordered list of IIDs for the i-th input table. A pinned row in the j-th input table with IID x defines a pinned space of $\prod_{i=1}^{j-1} R_i \times \{x\} \times \prod_{j=i+1}^{n} R_i$; i.e., the user is interested only in input combos with x participating. A pinned row in a result (intermediate or final) table defines a pinned space of $\{v \mid v \in \mathbf{R} \wedge x \text{ is a derivative row of } v\}$; i.e., the user is interested only in input combos that contribute to x. Considering Figure 2, the pinned space for the pinned o_1 is $\{\langle s_2, f_1 \rangle, \langle s_2, f_3 \rangle, \langle s_3, f_1 \rangle, \langle s_3, f_3 \rangle\}$ because these input combos contribute to the total revenue in o_1. If the user additionally pins f_1 in Frequents; this pinned row will have a pinned space of $\{s_0, \ldots, s_5\} \times \{f_1\}$. Thus, the overall pinned space will be shifted to $\{\langle s_2, f_1 \rangle, \langle s_3, f_1 \rangle\}$, meaning the user only wants to investigate how Dan contributes to Edge's total revenue.

Pinning has multiple uses. First, it augments I-Rex's tracing capability: pinning effectively allows *backward tracing* from a pinned

[1] Aggregate queries w/o GROUP BY are the same as having empty GROUP BY list.

result row produced by any stage to the pinned space of input combos, and then from there, forward tracing to result rows further downstream. Second, combined with stepping, pinning provides a form of *watchpointing*. With a pinned space in effect, I-Rex restricts stepping to the pinned space, effectively setting a watchpoint that pauses execution only at points relevant to the pinned rows. In Figure 2, with o_1 pinned, execution automatically stops at the 4 relevant input combos $\langle s_2, f_1\rangle$, $\langle s_2, f_3\rangle$, $\langle s_3, f_1\rangle$, $\langle s_3, f_3\rangle$, skipping irrelevant portions of execution before, after, and in between.

Note that the result IIDs across stages naturally encode provenance for efficient pinning. For example, although the IID of an aggregate result row does not encode its provenance by itself, the group-by values therein allow a group's how-provenance to be recovered by examining the IIDs for the preceding grouping stage (which contains these values too) and/or by querying.

3.2.4 Drilling Down and Pulling Up. As illustrated in Example 2, I-Rex allows the user to *drill down* into a subquery, analogous to "stepping into" a function call. Besides drilling down into subqueries in `WHERE`, `HAVING`, and `SELECT` expressions, I-Rex also allows drilling down into subqueries in `FROM`, which can be either directly nested therein or via a reference to some `WITH` definition. In these cases, the user can drill down directly through a particular row in a derived input table; I-Rex will open the debugging context for the subquery responsible for producing that table and automatically pin that row in the subquery's final output table. When debugging a complex query with many nested blocks, I-Rex essentially maintains a "call stack" of debugging contexts (similar to Figure 4). To let the user *pull up* from a subquery debugging context, I-Rex simply returns the user to the previous debugging context on the stack, which belongs to the enclosing query block. The state of the subquery debugging context is still preserved until the user changes the active input combo in the enclosing block's debugging context, which forces "stepping out" from the last subquery function call.

3.3 Supported Features and Limitations

I-Rex supports a rich set of SQL query features, such as `SELECT` queries, set/bag operations (e.g., `UNION`, `INTERSECT`, `EXCEPT`), subqueries (including correlated ones), outer joins and joins expressed in general `JOIN` syntax (except `LATERAL`), and `WITH`. I-Rex currently does not support recursive `WITH`, `LATERAL` joins and `WINDOW` functions.

However, I-Rex's debugging paradigm is very general. By modeling a query as a collection of syntactic blocks that act like function calls, any relational operator that deterministically transforms input tuples into output tuples can be adapted to this framework. Assigning totally ordered, logical identifiers (IIDs) allows I-Rex to enforce reproducible execution stages across complex operators. We briefly outline strategies for several possible future extensions: 1) for `WINDOW` functions, I-Rex could introduce a new execution stage where canonical execution performs a stable sort based on `PARTITION BY` and `ORDER BY` expressions; 2) `LATERAL` joins can be modeled in a way similar to correlated subqueries, where the left table acts as the caller passing bound parameters to the right table's context; 3) recursive `WITH` can potentially be supported by treating each recursive step as a context whose input tables are produced by the previous step, until a fixed point is reached.

Limitations. While I-Rex handles complex queries, it does have some limitations. First, as I-Rex relies on deterministic execution, it does not support built-in functions whose results cannot be reliably reproduced (e.g., `RAND`). The behavior of queries involving such functions may depend on the execution plan chosen by the optimizer, and cannot be reproducible by I-Rex's canonical execution.

Second, while features such as forward tracing and pinning allow a user to quickly examine execution around points of interest, I-Rex does not by itself suggest potential points of interest. Currently, I-Rex sorts and paginates all intermediate and final result tables in IID order, which makes it easier for the user to locate a row by its IID (or prefix thereof). A future extension is to support searching for specific result rows by an arbitrary filter condition, which can be efficiently implemented by issuing a rewritten query to compute only the filtered rows (in a similar vein as a *page-fetch query* in Section 4.1). A more intriguing extension would be to support pinning by condition instead of by row.

4 SYSTEM AND OPTIMIZATIONS

We now describe the implementation and optimizations of I-Rex. Given a query Q being debugged, a straightforward approach would be to carry out the canonical execution of Q and collect all debugging information in one go, but this approach is not scalable.

Example 3. *Consider a TPC-H [11] database instance generated with a scale factor of* 1 *(i.e., total size of all tables is* 1*GB), and the following subquery in the* `FROM` *clause of benchmark query Q8:*

```
|| SELECT EXTRACT(year from o_orderdate) as o_year,
||        l_extendedprice * (1 - l_discount) as volume,
||        n2.n_name as nation
|| FROM part, supplier, lineitem, orders, customer, nation n1, nation n2, region
|| WHERE p_partkey = l_partkey AND s_suppkey = l_suppkey
|| AND ... -- omitted for simplicity
```

If we were to compute and display the ten entire tables (8 input, 1 joined & filtered, 1 output) and lineage data, we would ship at least 2*GB of data from the database server to the I-Rex client, introducing unacceptable overhead over the network and client-side memory.*

To address this challenge, I-Rex uses three high-level ideas. (1) Instead of showing the entire canonical execution of Q, we let the user examine one small, relevant window of this execution at a time. (2) To obtain all debugging information needed for a particular execution window, instead of performing the canonical execution and instrumenting it, we can formulate SQL queries based on Q to compute such information directly and declaratively. (3) We can judiciously compute some summary data and then use it to further rewrite these queries to be more efficient.

As described in Section 2, I-Rex realizes idea (1) using a paginated display for each table. For each debugging context, the active input combo marks the current point of execution and controls which pages to display by default: pages containing the input combo for input tables, and the page containing the derivative row for each subsequent stage. Together, these pages define the "window" of execution seen by the user. The optimization of tracing and pinning builds on pagination; see the full version [36] for details.

4.1 Optimizing Pagination

Given a base or derived table to display, the potential savings of pagination are easy to see: by focusing on one page at a time,

	min_iid	s.bar	s.beer	f.drinker	f.bar
0	((Apex, Corona), (Amy, Apex))	[Apex, Edge]	[Amstel, Dixie]	[Amy, Ben]	[Apex, Edge]
1	((Edge, Amstel), (Dan, Edge))	[Edge, Edge]	[Amstel, Corona]	[Ben, Dan]	[Edge, Edge]
2	((Tavern, Amstel), (Coy, Tavern))	[Tavern, Tavern]	[Amstel, Erdinger]	[Coy, Coy]	[Tavern, Tavern]

Table 1: Milestone table for the joined & filtered table in Figure 2, with 3 pages and page size of 3 (rows). The IIDs have the format ((`s.bar`, `s.beer`), (`f.drinker`, `f.bar`)), and 3 min IIDs are those of the rows j_0, j_3, and j_6 respectively. The Bloom filter column is omitted.

we only need to compute, transmit, and render content on this page alone. The baseline solution to pagination supported by SQL uses its `OFFSET` and `LIMIT` features. However, `OFFSET` and `LIMIT` alone do not make queries faster (verified in Section 5). The optimizer typically has insufficient knowledge to skip directly to the `OFFSET`-th result row, so the query often executes from the very beginning to `OFFSET` + `LIMIT`, creating enormous waste. Furthermore, for queries that enforce result ordering (which is the norm in I-Rex as it aims to provide consistent and reproducible orderings for all results), simply determining the order would often involve computing and sorting all result rows, saving no computation cost.

To overcome this limitation, we observe that if we know which input rows contribute to the particular result page, we can use such information to prefilter the input rows and reduce execution cost. For example, in the joined & filtered table of Figure 2, suppose a user only needs to retrieve the page containing rows j_2 through j_5. It turns out that we only need $\{s_1, s_2\}$ from `Serves` and $\{f_1, f_3\}$ from `Frequents` to compute this page. In general, explicitly enumerating a set of such input rows is not scalable, but we can instead compute a compact summary of this set, at the expense of potentially introducing some false positives (but never any false negatives).

I-Rex has three types of summary-based filters, described further below, to cover the range of possibilities: 1) IID-based filters, 2) "sargable" [72] filters, and 3) Bloom filters [13]. All three require precomputing a summary of what input rows contribute to each page. This step, performed when the debugging context is initialized, executes a *milestone query* (genereated automatically by I-Rex) to produce a *milestone table* for each input table of the debugging context and for the result table of each stage. For each page of table contents, a milestone row contains the summaries of input rows that contribute to the page. The milestone tables are cached by the I-Rex client.[2] Subsequently, when a particular page of a table is requested, I-Rex consults the corresponding milestone table to generate a *page-fetch query* that incorporates filtering using the summaries pertaining to the page requested.

4.1.1 IID-Based Filtering. Recall that I-Rex sorts every table by its IID to ensure consistency and reproducibility. Hence, pages of a table partition its rows into consecutive, non-overlapping IID ranges. To enable IID-based filtering, we precompute, in the milestone table, the minimum IID among rows on each page. For example, Table 1 shows the milestone table for the joined & filtered table in Figure 2, with the minimum IID for each page captured by the `min_iid` column. Using this information, I-Rex can add a tight range condition on IID to the `WHERE` clause of a page-fetch query.

[2] I-Rex assumes the client sees a static snapshot of the underlying database; otherwise, data updates will invalidate the cached milestones.

Continuing with the same example, the following query fetches the contents of the second page of the joined & filtered table, while also synthesizing the IID for each result row. Note that the lower IID bound is the minimum IID of the second page in Table 1, and the (open) upper IID bound is the minimum IID of the next page:

```
SELECT *, ((s.bar,s.beer),(f.drinker,f.bar)) AS _iid -- synthesize IID
FROM Serves s, Frequents f
WHERE (...) -- original WHERE conditions
-- IID-based filtering:
AND ((s.bar,s.beer),(f.drinker,f.bar))>=(('Edge','Amstel'),('Dan','Edge'))
AND ((s.bar,s.beer),(f.drinker,f.bar))<(('Tavern','Amstel'),('Coy','Tavern'))
ORDER BY 1; -- order by IID
```

IID-based filtering serves two purposes. First, it rejects any result row not on the requested page. This feature is indispensable because the other types of filtering implemented may introduce false positives and admit rows outside the requested page; therefore, I-Rex always activates IID-based filtering to ensure correctness. Second, IID-based filtering can enable efficient execution. However, its potential is limited: from a range bound on a multi-component IID, we can safely infer a range bound only on the leading component, but not on subsequent components. For example, in the page-fetch query above, the query optimizer may infer that `s.bar` (and hence `f.bar` by transitivity) falls within [`Edge`, `Tavern`] and use an index on `s.bar` (or `f.bar`), but nothing is known about `s.beer` or `f.drinker`. This limitation motivates other types of filtering below.

4.1.2 Sargable Filtering. To enable efficient page-fetch queries, we aggressively look for opportunities to inject safe, *sargable* [72] predicates that enable index plans. To this end, for each column A in an input table, where an index already exists on A, I-Rex computes a concise summary of the A values, in the form of a single range [min, max], over all input rows that contribute to each page of the result table. For example, Table 1 shows the milestone table for the joined & filtered table in Figure 2, assuming that `s.bar`, `s.beer`, `f.drinker`, `f.bar` are the indexed columns. I-Rex injects a range condition for each in the `WHERE` clause of a page-fetch query.

Using the same example, the page-fetch query for the second page of the joined & filtered table can now be augmented as follows:

```
SELECT ... FROM Serves s, Frequents f
WHERE (...) -- original WHERE conditions
AND s.bar BETWEEN 'Edge' AND 'Edge' -- sargable filtering
AND s.beer BETWEEN 'Amstel' AND 'Corona'
AND f.drinker BETWEEN 'Ben' AND 'Dan'
AND f.bar BETWEEN 'Edge' AND 'Edge'
AND ... -- IID-based filtering
ORDER BY 1; -- order by IID
```

Since the sargable filters are always on indexed columns, they enable the optimizer to consider index plans that access only the relevant parts of the input tables. Even when index plans are not the most optimal, the inexpensive filter conditions still reduce the number of rows involved in downstream processing (e.g., join) and hence overall execution cost.

Pushing down filters too aggressively may have a negative effect when the filters are not selective, which can happen if the page size is not small and the result row ordering does not correlate with the filter column value. The optimizer may underestimate the output cardinality of the filter and choose a secondary index scan that is less efficient than a table scan. Therefore, we only inject a sargable filter for column A if its range on the requested page covers a small percentage of the entire domain of A. I-Rex uses 30% as a cutoff, which works well empirically.

Finally, instead of using one range to summarize input column values for a page, we can use multiple ranges to reduce false positives, at the expense of higher precomputation and storage costs for milestones. This trade-off is worth investigating as future work.

4.1.3 Bloom Filtering. Correlated subqueries in WHERE can still bottleneck query execution, especially if the external columns they reference are not indexed and therefore not covered by sargable filtering. One approach to avoid evaluating expensive correlated subqueries is memoization: regarding each correlated subquery q as a function, we can cache all value settings of the external columns that q is invoked with, along with the corresponding results returned by q. However, this approach is not scalable as the number of possible value settings can be high (even with sophisticated decorrelation techniques [73] to restrict such settings) and results can be large, requiring large cache spaces.

To balance space efficiency and performance, we take an approximate approach at a coarser grain: instead of capturing the behavior of each subquery precisely, we consider the entire WHERE condition (including any constituent subqueries), and use a Bloom filter [13] to track, for each page, the set of "relevant" input column values for which the entire WHERE evaluates to true. We choose relevant columns to be all input table column externally referenced by any subquery, plus any input table column involved in an atomic condition together with some subquery. For instance, in our running example (Q from Example 1), the only relevant input column is f.drinker. As another example, if the WHERE condition is A IN $(q(B))$, where q is a correlated subquery with external column reference B, the set of relevant input columns would be $\{A, B\}$. A single SQL query can conveniently precompute such Bloom filters using a user-defined aggregate, along with the rest of the milestone table.

Continuing with our running example, the page-fetch query for the second page of the joined & filtered table is now updated to:

```
|| SELECT ... FROM Serves s, Frequents f
|| WHERE (...) -- original WHERE conditions
|| AND ... -- sargable filtering
|| AND ... -- IID-based filtering
|| AND BLOOM_CHECK('10101010', f.drinker) -- Bloom filtering
|| ORDER BY 1; -- order by IID
```

Here, BLOOM_CHECK(F, e) is a user-defined function that checks if an entry e is in the Bloom filter F. Given the probabilistic design of Bloom filters, BLOOM_CHECK may return false positives but not false negatives. '10101010' is the Bloom filter computed for the second page, encoding all f.drinker values over input combos that contribute to this page. Failing BLOOM_CHECK allows query execution to save time by bypassing the evaluation of the expensive subquery. In this example, row f_2, with f.drinker of Coy, fails the subquery condition, so the computed Bloom filter will not have the contribution of Coy. Hence, when evaluating the above page-fetch query, even though Coy still passes the sargable filter, it will fail BLOOM_CHECK, skipping rest of WHERE including the subquery. Yet, because of false positives, passing BLOOM_CHECK does not mean WHERE must evaluate to true; the original WHERE condition still must be included.

Proposition 4.1. *Suppose functions* BLOOM_GEN : $\mathcal{P}(tuples) \rightarrow bitstrings$ *and* BLOOM_CHECK : $bitstrings \times tuples \rightarrow \{true, false\}$ *satisfy* $e \in \mathcal{V} \Rightarrow$ BLOOM_CHECK(BLOOM_GEN$(\mathcal{V}), e)$ *for any tuple* e *and any set* $\mathcal{V}$ *of tuples of the same sort. Let* $\{R_i\}$ *denote a list of (potentially aliased) input tables and* Θ *a condition over* $\{R_i\}$*. For any subset* $\mathcal{A}$ *of columns in* $\{R_i\}$*, the following two queries are equivalent:*

Q_1: SELECT * FROM $\ldots, R_i, \ldots$ WHERE Θ;

Q_2: SELECT * FROM $\ldots, R_i, \ldots$ WHERE Θ AND BLOOM_CHECK(F, $\langle \mathcal{A} \rangle$);
where F = BLOOM_GEN(SELECT $\langle \mathcal{A} \rangle$ FROM $\ldots, R_i, \ldots$ WHERE Θ).

The proposition above implies considerable freedom in choosing the set of relevant input columns to track for a Bloom filter. Instead of tracking the IIDs of the input rows (which can be many), we decided to track input column values that influence the outcome of evaluating conditions involving subqueries, because there are usually far fewer distinct values to track. This heuristic has worked well for I-Rex. Finally, Bloom filter conditions are not sargable and cannot be used to avoid full table scans. Evaluating them introduces overhead. Therefore, I-Rex uses Bloom filtering only for "short-circuiting" evaluation of expensive WHERE clauses with subqueries. If a Bloom filter returns too many false positives, the overhead of BLOOM_CHECK may outweigh the savings achieved by skipping subquery evaluation. I-Rex automatically estimates the false positive rate of each Bloom filter and inject the BLOOM_CHECK condition only if this rate is less than 50%.

4.1.4 Summary. I-Rex optimizes pagination by generating page-fetch queries that automatically combine three types of summary-based filters to prune unnecessary data. First, IID-based filtering uses the minimum IID of a page to create a precise page boundary; it is *always* applied to guarantee correctness by rejecting rows outside the requested page. Second, sargable filtering injects dynamic range bounds for indexed columns to encourage efficient index scans. Sargable filters are available only when the query block has base tables with indexed columns, and they are automatically applied on top of IID filters. Finally, Bloom filtering tracks relevant input column values, skipping the evaluation of expensive correlated subqueries. I-Rex proactively detects the existence of correlated subqueries and automatically computes and applies Bloom filters along with IID and sargable filters whenever possible.

4.2 System Overview

We briefly describe the overall I-Rex system while leaving most implementation and optimization details to the full version [36].

I-Rex operates a highly scalable, stateless client-middleware-server architecture.The frontend client is built with React, which handles user interaction logic, visualizes the debugging contexts, and manages the caching of data and SQL code to ensure a smooth, interactive experience. The middleware server performs the heavy lifting of query analysis and rewriting using Apache Calcite [10]; it "compiles" the original query to be debugged into a collection of rewritten SQL query templates to support debugging. Each debugging action initialized by the client is supported by executing appropriately instantiated query templates on the underlying database. I-Rex extends the database server using user-defined functions (e.g., for Bloom filters) but does not modify its internals otherwise.

Importantly, to support many concurrent users without creating bottlenecks, I-Rex maintain no session-specific state for any active debugging session in the middleware or the underlying database. When a client starts a debugging session, the middleware performs a compilation step to analyzes the original query. The query is divided into logical blocks (Figure 4), and for each block (which will

produce a debugging context when invoked at runtime), the middleware automatically generates milestone queries, page-fetch queries, and queries supporting other debugging features, as parameterized SQL templates. In the step, the middleware also employs several equivalent query rewrites (including scalar subquery optimization, sargable filter injection, and recursive filter pushdown, further described in the full version [36]) to help enable optimizations that tend to be missed by the underlying database optimizer. Finally, the result of this compilation step is shipped to and cached in the client. As a user enters a specific debugging context, the client initializes the context by executing the cached milestone queries for the corresponding block and caching the result milestone tables. Subsequently, as the user navigates within the debugging context, the client dynamically instantiates page-fetch and other supporting queries on demand using the cached milestone tables, enabling fast exploration without computing the full query.

5 PERFORMANCE EXPERIMENTS

We conduct experiments to evaluate the performance and scalability of I-Rex under the optimizations discussed in Section 4. We focus on evaluating page-fetch queries (Section 4.1) and milestone computation, as they are the most expensive queries that I-Rex uses; other operations (e.g., tracing and pinning) either use page-fetch queries or relatively cheap queries with highly selective conditions. The baseline for fetching pages is to use SQL `OFFSET` and `LIMIT`. We enable various pagination optimizations in I-Rex to evaluate their respective benefits. IID-based filtering is always enabled (for correctness). Section 5.1 evaluates pagination with Bloom filtering but not other optimizations, while Section 5.2 evaluates pagination with sargable filtering but not others. Finally, Section 5.3 enables all pagination optimizations and evaluates both the performance of pagination and milestone computation.

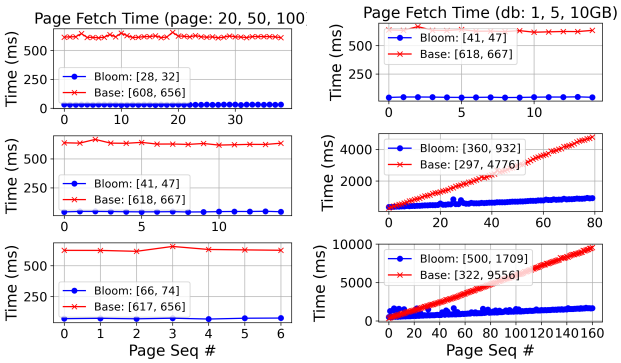
We use the TPC-H benchmark [11] and generate 3 database instances of sizes 1GB (benchmark default), 5GB, and 10GB. In addition to the default indexes on the primary keys, we also create a reasonable set of secondary indexes (details in [36]) that simulate typical usage. For Sections 5.1 and 5.2, we show one query for each, where the respective optimization is applicable and can be best evaluated; for the general evaluation in Section 5.3, we show results for all 22 benchmark queries. All experiments were done on a 64-bit Ubuntu 22.04 LTS server with four Intel(R) Xeon(R) 6530P CPUs @ 2.30GHz, 64GB RAM, and 256GB disk space. We used PostgreSQL 18.3 [67] with `work_mem=128MB` and `shared_buffers=8GB`; we also turned off parallel execution (`max_parallel_workers_per_gather=0`) to reduce its potentially confounding effect, so results are easier to interpret.

5.1 Effectiveness of Bloom Filtering

This experimental setup uses a variant of TPC-H Q2:

```
SELECT ROW(p_partkey, s_suppkey, ps_partkey, ps_suppkey,
       n_nationkey, r_regionkey), *
FROM part, supplier, partsupp, nation, region
WHERE ... AND ps_supplycost = (
    SELECT MIN(ps_supplycost)
    FROM partsupp, supplier, nation, region
    WHERE p_partkey = ps_partkey -- p_partkey from outer query
    AND ...);
```

As discussed in Section 4.1, I-Rex precomputes a Bloom filter for the `p_partkey` values for input rows that contribute to each result page,



(a) *Varying page size; 1GB database.* **(b)** *Varying database size; 50-row pages.*
Figure 5: Bloom filtering vs. baseline: time to fetch each page. $[\cdot, \cdot]$ in legend shows min/max page fetch times.

and when fetching a page, uses the Bloom filter to short-circuit the evaluation of the rest of `WHERE` containing an expensive correlated subquery. However, the bloom filter is only used for membership checking if the false positive rate is less than 50%.

By default, we set the number of Bloom filter bits to $m = 1024$; we conservatively estimate the number of unique `p_partkey` in each page (denoted by n) as the page size; accordingly, we set the number of Bloom filter hash functions to $\frac{m}{n} \times \ln 2$.

We conduct two sets of experiments. First, we fix the database size to 1GB and vary the page size. Second, we fix the page size to 50 and vary the database size. We compare page-fetch queries with Bloom filtering with the baseline using `OFFSET` and `LIMIT`. We collect the execution times reported by the `EXPLAIN ANALYZE` command for all queries and show them in Figure 5.

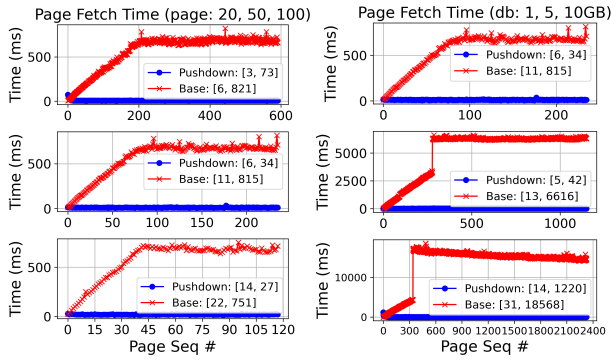
Overall, the results show that Bloom filtering is very effective for Q2. When the database size is relatively small (Figure 5a), the baseline consistently takes around 600ms to reproduce a page, because its execution cost is dominated by the computation of the entire result and sorting them first; in contrast, Bloom filtering takes 28 to 74ms, with larger pages requiring more time. Under larger database sizes (Figure 5b), baseline switches to a different plan whose cost grows linearly with the starting position of the page fetch, eventually matching the cost of executing the entire query when fetching pages near the tail; Bloom filtering performance remains scalable and depends much less on the fetch position, saving as much as 3.8s (5.1× speedup) and 7.8s (5.6× speedup) for 5GB and 10GB databases, respectively.

5.2 Effectiveness of Sargable Filtering

This setup uses the joined & filtered table for benchmark Q7:

```
SELECT ROW(s_suppkey, l_orderkey, l_linenumber, o_orderkey,
       c_custkey, n1.n_nationkey, n2.n_nationkey), *
FROM supplier, lineitem, orders, customer, nation n1, nation n2
WHERE ...; -- same as original query
```

We create sargable range filters for all index columns in the input tables based on the precomputed milestones, but only if the ranges cover no more than 30% of the domain, as discussed in Section 4.1. We run two sets of experiments similar to those for Bloom filtering, with results shown in Figure 6. Overall, we observe that the baseline using `OFFSET` and `LIMIT` performs progressively worse when it fetches later pages, until its running time plateaus when the plans switch to essentially computing and sorting the entire result; in contrast,



(a) *Varying page size; 1GB database.* **(b)** *Varying database size; 50-row pages.*
Figure 6: Sargable filtering vs. baseline: time to fetch each page. [·, ·] in legend shows min/max page fetch times.

	1GB (ms)		5GB (ms)		10GB (ms)	
Page	I-Rex	Baseline	I-Rex	Baseline	I-Rex	Baseline
head	2,532.81	5,870.729	14,184.028	46,234.889	37,419.067	120,576.875
middle	2,456.265	7,991.925	16,215.405	55,316.965	32,434.602	133,646.155
tail	2,787.514	10,113.122	13,983.933	64,399.041	33,393.882	146,715.434

Table 2: I-Rex vs. baseline: time to fetch a page in the joined & filtered table of Q18; 50-row pages and varying database size. Here, head refers to the first query while tail refers to the second to the last (as the last page sometimes is not full).

sargable filtering performs well across all pages regardless of their position, and its advantage over the baseline widens dramatically toward later pages. With a 1GB database (Figure 6a), baseline can take up to 821ms to fetch a page, while sargable filtering takes no more than 73ms. Fixing the page size at 50 and using larger databases (Figure 6b), sargable filtering's benefit is even greater. For the 10GB database, the baseline takes about 19s to fetch a page positioned at 1/6-th of the entire result or later, while sargable filtering takes about 1.2s (16× speedup) in the worst case.

5.3 General Evaluation

For general evaluation, we apply all I-Rex optimizations to all TPC-H queries, and compare the execution times of the resulting page-fetch queries with those of the baseline approach. IID-based and sargable filtering can be applied to all queries, while Bloom filtering is only applicable to Q4, Q16, Q18, and Q20–22. Because of limited space, we only show in Table 2 the results for the joined & filtered table of Q18, which is the most expensive query in our experiments; remaining results are in [36]. The results generally echo the findings from the experiments on Bloom/sargable filtering, and confirm the benefit of combining optimizations. Specifically in Table 2, I-Rex performs no worse than the baseline for pages at the beginning of the results, and significantly better for later pages.

Finally, we evaluate the overhead of debugging context initialization, which is dominated by the cost of running milestone queries. We compare their costs with those of computing the entire results of the corresponding original query blocks. Because of limited space, we only show in Table 3 the results for the three most expensive milestone/original query pairs, with a 1GB database and 50-row pages; the remaining results are in [36]. All three pairs come from Q18's stages. Since milestone queries conceptually summarize the output of the original queries, we do not expect them to run faster than the latter. As Table 3 shows, their performance is comparable

	Milestone Query		Original Query	
Q18	Exec. time (ms)	Output (MB)	Exec. time (ms)	Output (MB)
joined & filtered	15,984.578	52	12,086.886	1272
group	13,193.212	9	12,602.33	64
output	13,808.861	9	14,452.551	75

Table 3: Milestone vs. original queries: execution time and output size; 50-row pages and 1GB database.

to the original queries. Recall from earlier experiments that baseline page-fetch queries often cost as much as the original queries, this observation implies that I-Rex can benefit from milestone-enabled optimizations for many page-fetch queries by paying only a one-time overhead equivalent to a single baseline page-fetch query. The execution times reported are taken from `EXPLAIN ANALYZE`, which only measures the time spent in executing the query but not transmitting its result. We intentionally chose to exclude the latter because the outputs from the original queries can be too large to transmit. For example, printing the entire Q18's joined & filtered table took more than an hour for a `psql` client running on the database console. In contrast, as Table 3 shows, milestone queries return much smaller results, which are feasible to transmit to and handle by the client.

The end-to-end latency of I-Rex consists of time for page-fetch query execution, network transmission, and frontend rendering. While query times naturally vary and network latency fluctuates unpredictably, the frontend rendering is consistently lightweight. On average, rendering a 50-tuple page takes ~600ms, forward tracing ~300ms, and pinning ~500ms, providing an interactive experience.

Limitations and Opportunities. While I-Rex aims at making SQL debugging interactive, for queries that are inherently hard to optimize and expensive to evaluate, debugging them remains challenging. For example, the latency in debugging context initialization is dominated by milestone queries, which cannot be expected to outperform the original query (other than transmitting less result data). Possible directions to increase interactivity include on-demand and/or approximate milestone computation, which are promising venues for future research. An orthogonal approach is to find small database instances that can help reveal potential mistakes in the query being debugged. Sampling is one method, but smarter query-driven methods have also been studied; see Section 7 for more discussion. This approach complements I-Rex by reducing the size of the database used to debug in the first place.

6 USER STUDY

We conducted a user study in an undergraduate database course to evaluate the effectiveness of I-Rex on two aspects: (1) whether I-Rex helps users catch more logical bugs, and (2) whether I-Rex reduces the time to find bugs.

Participants. We had 237 student participants who had just become familiar with SQL at the time of the study. Participation was voluntary. We considered recruiting participants from other sources (e.g., Amazon Mechanical Turk), but decided against it as it was hard to ensure participants' SQL familiarity was at a similar level. Since SQL familiarity significantly impacts debugging time, the lack of control can make results difficult to interpret.

Preparation and Setup. The study was conducted for two consecutive weeks during the course's once-a-week 75-minute discussion sessions in the Fall 2022 semester. In the first session, students were given a tutorial on I-Rex and informed about the format of the quiz, which contained two SQL debugging problems P1 and P2. After the first discussion, I-Rex is made public for students to try. In the

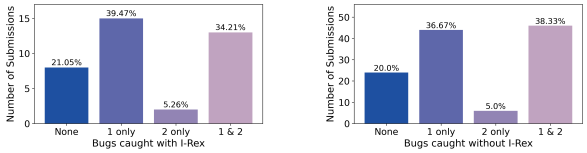


Figure 7: Bugs caught (out of two) for P1, using I-Rex vs. not.

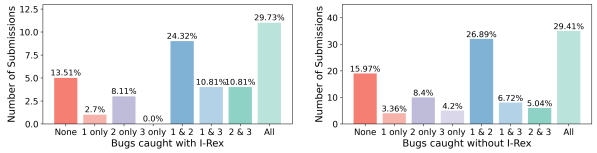


Figure 8: Bugs caught (out of three) for P2, using I-Rex vs. not.

second session, students completed the quiz synchronously in a proctored environment, where they were asked not to discuss with classmates. For each problem in the quiz, students were provided a problem statement, an incorrect query, its incorrect output, and the correct output (details in [36]). To create treatment and control groups, students received the two problems in a random order. For the first problem they received, they were free to use any tool of their choice (e.g., Gradescope autograder, `psql` console, `pgAdmin` Web interface) except I-Rex. For the second problem, I-Rex was made *available but optional* along with other tools. This user study was conducted before any AI tool was released; thus, no AI assistant was available to students. For each problem, students were to describe the bugs found in a free-response text box. P1 had two bugs and P2 had three, and the students were not told how many. While students self-paced, they were recommended to spend 15-20 minutes on each problem. They were not allowed to move on to the second problem until they submitted an answer to the first problem.

Results and Analysis. We collected the time it took students to solve each problem, whether they chose to use I-Rex, and the bugs they found. Of 237 students, 140 completed both problems and provided legitimate answers. Therefore, we based our analysis on these 140 responses. For the first problem received, which must be completed without I-Rex, 73 students received P1, and 67 received P2. For the second problem received, where using I-Rex was an option, 73 students received P2, but 36 of them did not use I-Rex; 67 students received P1, but 29 of them did not use I-Rex. In summary, for P1, we have 38 submissions using I-Rex, and 102 not using I-Rex; for P2, we have 37 submissions using I-Rex, and 103 not.

We manually reviewed each response and checked whether the student correctly identified the bugs. Figure 7 shows the fraction of the submissions that correctly identified each possible subset of the bugs for P1, with (left) or without (right) help from I-Rex; Figure 8 does the same for P2. While the total submission numbers differ between using/not using I-Rex, the distributions (fractions for possible subsets) are similar. The use of I-Rex had no discernible impact on the mean of bugs identified, as shown in the last column of Table 4. While I-Rex did not seem to help students find more bugs, the comparison of debugging times, reveals that the use of I-Rex reduces the mean debugging time by around 8 minutes for both problems. We further prove I-Rex's efficiency through statistical tests. Due to the non-normal distribution of the collected time data (tested by the Shapiro-Wilk Test [74]), we performed the Mann-Whitney U Test [60] for P1 and P2 separately. With the null hypothesis being "I-Rex makes no difference or slows students down", the p-values for P1 and P2 are 0.0001 and 0.0007, respectively. Since they are below the standard 0.05 threshold, we reject the null hypothesis and conclude that I-Rex significantly improves students' efficiency in finding bugs without compromising accuracy.

Student Feedback. Students also provided anonymous feedback on the use of I-Rex. A common theme is that I-Rex has a noticeable learning curve: since it exposes step-by-step logical execution and displays numerous visual elements simultaneously, some students initially found it intimidating. However, those who pushed past this familiarization phase found it a powerful and effective debugging aid. There were also students who preferred using traditional clients (like `pgweb`) to manually construct auxiliary queries, highlighting that debugging preferences remain highly subjective and tied to individual cognitive styles. Just like programmers have different preferences for debugging in general (from printing to console to sophisticated IDE-supported execution tracing and state inspection), we envision I-Rex not as a one-size-fit-all SQL debugging solution, but as a useful addition to the SQL debugging toolbox.

Discussion. There are several limitations to this user study. First, its participation was voluntary with no extrinsic incentives, so students might not feel compelled to invest the upfront time needed to overcome the tool's learning curve. This likely explains why many opted against using I-Rex for the second problem. Besides the lack of incentives, the strict time constraints may have also reduced their overall debugging effort. To address this, future evaluations should develop better incentives for participants to fully acclimate.

Second, limited by time and subject pool, this study only evaluated undergraduates on pedagogical queries. Future work is needed to assess the tool's effectiveness among industry professionals debugging complex queries over enterprise-scale databases. Such settings may also help answer the question of whether I-Rex can help users find new, harder bugs, beyond making bug finding faster.

Third, this study does not explore the use of large language models (LLMs) as a potential alternative or augmentation to I-Rex. The fast evolution of LLMs is rapidly changing how SQL queries are composed and debugged. While many text-to-SQL frameworks [37, 39, 58, 68, 69, 77] have shown good accuracy on benchmarks such as BIRD [59] and Spider 2.0 [56], query debugging remains a major challenge as shown in [83]. To investigate the increasingly common use of LLMs for SQL debugging, we conducted a small case study (details in [36]) by feeding the incorrect queries from our user study to an LLM. Despite the relative simplicity of these educational queries, the LLM still occasionally misidentified bugs or hallucinated incorrect interpretations. Because LLMs lack formal correctness guarantees and produce inherently non-deterministic outputs, conducting a direct, reproducible comparison is difficult. We postulate that LLMs and I-Rex serve orthogonal purposes: LLMs act as probabilistic oracles designed to instantly suggest a patch, whereas I-Rex is an interactive tool for investigating logical execution to understand query behavior under unforeseen circumstances. Rather than competing, I-Rex provides the deterministic ground truth that LLMs currently lack. Specifically, future LLM-driven agents could directly call I-Rex backend API server to examine intermediate query execution states accurately and efficiently, thereby significantly improving their reliability.

Problem	# response	Mean time	Mean bugs found
P1 w/ I-Rex	38	12.85 min	1.13 / 2
P1 w/o I-Rex	102	20.59 min	1.18 / 2
P2 w/ I-Rex	37	19.12 min	1.91 / 3
P2 w/o I-Rex	103	27.63 min	1.85 / 3

Table 4: Mean debugging time and bugs caught per problem.

7 RELATED WORK

Earlier versions of I-Rex [47, 61] precomputed and stored all debugging information in the client, which did not scale. The version described in this paper has refined existing features and introduced new ones, and most importantly, added scalability support, which requires a redesign of the system and new optimization techniques.

Finding Logical Errors in Queries. Work toward finding logical errors in queries can be classified into two categories. The first category assumes knowledge of a correct reference query and uses it to help identify errors in an incorrect query. XData [25] checks the correctness of a query by running the query on self-generated testing datasets. Cosette [27–29], SQLSolver [34], and QED [80] test query equivalence using constraint solvers and theorem provers. RATest [62] aims at constructing small and illustrative database instances to show the differences between two queries. [24] develops a grading system that canonicalizes queries with rewrite rules and then decides query similarity using edit distance between the resulting logical plans. SQLRepair [70] and QR-Hint [46] focus on fixing the wrong query by proposing syntactical edits. I-Rex differs from this line of work as it assumes no reference query. Dropping this assumption fundamentally changes the problem and makes existing solutions inapplicable in more general settings, although they can complement I-Rex when reference queries are available.

The second category of work helps users examine a query without a given reference query. Qex [79] generates input relations and parameter values for unit-testing SQL queries. SQLLint [14–16] looks for patterns in the query indicative of common semantic errors and alerts users to them. C-instances [40] are abstract instances aimed at illustrating different ways a query can be satisfied. Interactive query builders and visualizers [1–5, 23, 44, 51, 57, 63] use diagrams to gain intuitive understanding of query logic. Frameworks for data exploration [5, 7, 33, 54] use signals such as query history and user feedback to suggest queries, which may serve as a debugging approach. A line of work known as algorithmic debugging [19, 20] guides users through a series of questions on whether intermediate query steps produce intended results. Also useful to debugging is explaining what queries do in natural language [17, 38, 50, 53, 75, 81]. None of the above supports debugging by tracing execution, as I-Rex and most GPL debuggers do.

Two systems in this second category, DESQL [45] and Habitat [32, 43], are the closest to I-Rex in approach. DESQL [45] is a debugger for Spark SQL, which decomposes the query into subqueries and helps users examine subqueries' output; it does not support correlated subqueries. At a conceptual level, DESQL fundamentally differs from our work because it adopts an operator-based view of execution that is closer to optimized execution plans than to the way SQL queries are written. Technical approaches also differ significantly: DESQL relies on Spark as the execution backend instead of traditional database systems, and assumes that debugging commences only after the instrumented query fully executes. Habitat [32, 43] allows users to mark SQL subexpressions and inspect intermediate results side by side, connecting related result rows. It also lets users filter these results to focus on a subset. However, Habitat differs from I-Rex in important ways. First, on scalability, I-Rex avoids computing full results of a query, while Habitat executes a query in full and shows its results in bulk. Although Habitat's focus filters can restrict the query, they need to be defined manually and explicitly; in contrast, I-Rex's pinning and automatic pagination are intuitive and demand less user effort, and I-Rex has optimizations to improve query efficiency. Second, I-Rex and Habitat have different conceptual designs: I-Rex defines its canonical query execution to be row-oriented and reproducible (including row ordering), while Habitat presents a set-oriented view of SQL execution. Finally, Habitat is not publicly available for comparison.

Other Areas of Related Work. Several areas of research are related to some of the ideas used by I-Rex. First, tracing and pinning in I-Rex use information about data provenance [12, 18, 26, 42, 48], which explains why and how a particular row is or is not produced by a query. Many previous works [6, 8, 9, 30, 31, 41, 49, 52, 55, 71] have focused on making provenance capture a built-in feature in database systems. In particular, [64] proposes techniques for capturing provenance by rewriting SQL queries, just as I-Rex uses query rewriting to help support tracing and other debugging operations. However, [64] captures full provenance information for all query results. In contrast, for scalability and interactivity, I-Rex computes provenance information on demand to avoid the overhead of full computation and materialization.

Second, efficient pagination of query results has been studied in query optimization literature. Recent advances have been made on direct access to query answers [21, 22, 35, 78], but their techniques require specialized indexes and apply only to conjunctive queries, less general than what I-Rex supports. Work on data skipping and predicate pushdown [65, 66, 76, 82] also requires significant changes to database system internals, but I-Rex chooses to leverage existing database systems for efficient pagination. I-Rex's design also motivates unique optimizations, such as client-cached milestones.

8 CONCLUSION

In this paper, we have presented I-Rex, a novel, interactive, scalable, and easy-to-deploy SQL debugger that, given a database instance, enables users to visualize the logical execution of SQL queries and thus debug query semantics in a GPL style. I-Rex executes a query in a canonical fashion that faithfully follows how the query is written, and lets users examine the execution using a rich set of features, including but not limited to those found in GPL debuggers. I-Rex supports efficient exploration of any arbitrary point of execution without fully executing the underlying query. It does so by fully leveraging the database system: it formulates the selective computation of debugging information around the point of interest as SQL queries and employs materialization and query rewrite strategies to ensure efficient execution. Our performance experiments and user study demonstrate the efficiency and effectiveness of I-Rex.

For future work, it is worth investigating milestone designs beyond single ranges and Bloom filters, and adding more debugging operations, such as setting watchpoints to observe filter conditions in actions. More features can be developed to help debug runtime SQL errors (such as division by zero), for which database systems usually provide uninformative feedback.

REFERENCES

[1] 2023. dbForge. https://www.devart.com/dbforge/mysql/querybuilder/.

[2] 2023. Microsoft Access. https://www.microsoft.com/en-us/microsoft-365/access.

[3] 2023. PgAdmin. https://www.pgadmin.org/.

[4] 2023. Rapid SQL. https://www.idera.com/rapid-sql-ide/.

[5] Azza Abouzied, Joseph Hellerstein, and Avi Silberschatz. 2012. Dataplay: interactive tweaking and example-driven correction of graphical database queries. In *Proceedings of the 25th annual ACM symposium on User interface software and technology*. 207–218.

[6] Parag Agrawal, Omar Benjelloun, Anish Das Sarma, Chris Hayworth, Shubha Nabar, Tomoe Sugihara, and Jennifer Widom. 2006. Trio: A system for data, uncertainty, and lineage. In *VLDB*, Vol. 6. 1151–1154.

[7] Javad Akbarnejad, Gloria Chatzopoulou, Magdalini Eirinaki, Suju Koshy, Sarika Mittal, Duc On, Neoklis Polyzotis, and Jothi S Vindhiya Varman. 2010. SQL QueRIE recommendations. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 1597–1600.

[8] Yael Amsterdamer, Susan B Davidson, Daniel Deutch, Tova Milo, Julia Stoyanovich, and Val Tannen. 2011. Putting lipstick on pig: Enabling database-style workflow provenance. *arXiv preprint arXiv:1201.0231* (2011).

[9] Bahareh Sadat Arab, Su Feng, Boris Glavic, Seokki Lee, Xing Niu, and Qitian Zeng. 2018. GProM-a swiss army knife for your provenance needs. *A Quarterly bulletin of the Computer Society of the IEEE Technical Committee on Data Engineering* 41, 1 (2018).

[10] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data*. 221–230.

[11] TPC Benchmark. [n.d.]. http://www.tpc.org/tpch.

[12] Nicole Bidoit, Melanie Herschel, and Katerina Tzompanaki. 2014. Query-based why-not provenance with nedexplain. In *Extending database technology (EDBT)*.

[13] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Commun. ACM* 13, 7 (1970), 422–426. https://doi.org/10.1145/362686.362692

[14] Stefan Brass and Christian Goldberg. 2004. Detecting Logical Errors in SQL Queries. In *Tagungsband zum 16. GI-Workshop Grundlagen von Datenbanken, Mohnheim, NRW, Deutschland, 1.-4. Juni 2004*, Mireille Samia and Stefan Conrad (Eds.). Universität Düsseldorf, 28–32.

[15] Stefan Brass and Christian Goldberg. 2005. Proving the Safety of SQL Queries. In *Fifth International Conference on Quality Software (QSIC 2005), 19-20 September 2005, Melbourne, Australia*. IEEE Computer Society, 197–204. https://doi.org/10.1109/QSIC.2005.50

[16] Stefan Brass and Christian Goldberg. 2006. Semantic errors in SQL queries: A quite complete list. *J. Syst. Softw.* 79, 5 (2006), 630–644. https://doi.org/10.1016/J.JSS.2005.06.028

[17] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[18] Peter Buneman, Sanjeev Khanna, and Tan Wang-Chiew. 2001. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001: 8th International Conference London, UK, January 4–6, 2001 Proceedings 8*. Springer, 316–330.

[19] Rafael Caballero, Yolanda García-Ruiz, and Fernando Sáenz-Pérez. 2012. Algorithmic debugging of SQL views. In *Perspectives of Systems Informatics: 8th International Andrei Ershov Memorial Conference, PSI 2011, Novosibirsk, Russia, June 27-July 1, 2011, Revised Selected Papers 8*. Springer, 77–85.

[20] Rafael Caballero, Yolanda García-Ruiz, and Fernando Sáenz-Pérez. 2012. Declarative debugging of wrong and missing answers for SQL views. In *Functional and Logic Programming: 11th International Symposium, FLOPS 2012, Kobe, Japan, May 23-25, 2012. Proceedings 11*. Springer, 73–87.

[21] Nofar Carmeli, Nikolaos Tziavelis, Wolfgang Gatterbauer, Benny Kimelfeld, and Mirek Riedewald. 2023. Tractable Orders for Direct Access to Ranked Answers of Conjunctive Queries. *ACM Trans. Database Syst.* 48, 1 (2023), 1:1–1:45. https://doi.org/10.1145/3578517

[22] Nofar Carmeli, Shai Zeevi, Christoph Berkholz, Benny Kimelfeld, and Nicole Schweikardt. 2020. Answering (Unions of) Conjunctive Queries using Random Access and Random-Order Enumeration. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2020, Portland, OR, USA, June 14-19, 2020*, Dan Suciu, Yufei Tao, and Zhewei Wei (Eds.). ACM, 393–409. https://doi.org/10.1145/3375395.3387662

[23] Claudio Cerullo and Marco Porta. 2007. A system for database visual querying and query visualization: Complementing text and graphics to increase expressiveness. In *18th International Workshop on Database and Expert Systems Applications (DEXA 2007)*. IEEE, 109–113.

[24] Bikash Chandra, Ananyo Banerjee, Udbhas Hazra, Mathew Joseph, and S. Sudarshan. 2021. Edit Based Grading of SQL Queries. In *CODS-COMAD 2021: 8th ACM IKDD CODS and 26th COMAD, Virtual Event, Bangalore, India, January 2-4, 2021*, Jayant R. Haritsa, Shourya Roy, Manish Gupta, Sharad Mehrotra, Balaji Vasan Srinivasan, and Yogesh Simmhan (Eds.). ACM, 56–64. https://doi.org/10.1145/3430984.3431012

[25] Bikash Chandra, Bhupesh Chawda, Biplab Kar, K. V. Maheshwara Reddy, Shetal Shah, and S. Sudarshan. 2015. Data generation for testing and grading SQL queries. *VLDB J.* 24, 6 (2015), 731–755. https://doi.org/10.1007/S00778-015-0395-0

[26] Adriane Chapman and HV Jagadish. 2009. Why not?. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 523–534.

[27] Shumo Chu, Brendan Murphy, Jared Roesch, Alvin Cheung, and Dan Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *Proc. VLDB Endow.* 11, 11 (2018), 1482–1495. https://doi.org/10.14778/3236187.3236200

[28] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An Automated Prover for SQL. In *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf

[29] Shumo Chu, Konstantin Weitz, Alvin Cheung, and Dan Suciu. 2017. HoTTSQL: proving query rewrites with univalent SQL semantics. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 510–524. https://doi.org/10.1145/3062341.3062348

[30] Yingwei Cui, Jennifer Widom, and Janet L Wiener. 2000. Tracing the lineage of view data in a warehousing environment. *ACM Transactions on Database Systems (TODS)* 25, 2 (2000), 179–227.

[31] Ralf Diestelkämper and Melanie Herschel. 2020. Tracing nested data with structural provenance for big data analytics.. In *EDBT*. 253–264.

[32] Benjamin Dietrich and Torsten Grust. 2015. A SQL debugger built from spare parts: Turning a SQL: 1999 database system into its own debugger. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 865–870.

[33] Kyriaki Dimitriadou, Olga Papaemmanouil, and Yanlei Diao. 2014. Explore-by-example: An automatic query steering framework for interactive data exploration. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 517–528.

[34] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proc. ACM Manag. Data* 1, 4 (2023), 227:1–227:26. https://doi.org/10.1145/3626768

[35] Idan Eldar, Nofar Carmeli, and Benny Kimelfeld. 2024. Direct Access for Answers to Conjunctive Queries with Aggregation. In *27th International Conference on Database Theory, ICDT 2024, March 25-28, 2024, Paestum, Italy (LIPIcs)*, Graham Cormode and Michael Shekelyan (Eds.), Vol. 290. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 4:1–4:20. https://doi.org/10.4230/LIPICS.ICDT.2024.4

[36] Full version of this submission. [n.d.]. https://github.com/yihaoh/irex_evaluation. ([n. d.]).

[37] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145.

[38] Sebastian Gehrmann, Falcon Dai, Henry Elder, and Alexander Rush. 2018. End-to-End Content and Plan Selection for Data-to-Text Generation. In *Proceedings of the 11th International Conference on Natural Language Generation*. Association for Computational Linguistics, Tilburg University, The Netherlands, 46–56. https://doi.org/10.18653/v1/W18-6505

[39] Pushpendu Ghosh, Aryan Jain, and Promod Yenigalla. 2025. SQLGenie: A Practical LLM based System for Reliable and Efficient SQL Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, Georg Rehm and Yunyao Li (Eds.). Association for Computational Linguistics, Vienna, Austria, 1004–1012. https://doi.org/10.18653/v1/2025.acl-industry.71

[40] Amir Gilad, Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2022. Understanding Queries by Conditional Instances. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 355–368. https://doi.org/10.1145/3514221.3517898

[41] Boris Glavic and Gustavo Alonso. 2009. Perm: Processing provenance and data on the same data model through query rewriting. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 174–185.

[42] Todd J Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance semirings. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 31–40.

[43] Torsten Grust and Jan Rittinger. 2013. Observing sql queries in their natural habitat. *ACM Transactions on Database Systems (TODS)* 38, 1 (2013), 1–33.

[44] Laura M Haas, Johann Christoph Freytag, Guy M Lohman, and Hamid Pirahesh. 1989. Extensible query processing in Starburst. In *Proceedings of the 1989 ACM SIGMOD international conference on Management of data*. 377–388.

[45] Sabaat Haroon, Chris Brown, and Muhammad Ali Gulzar. 2024. DeSQL: Interactive Debugging of SQL in Data-Intensive Scalable Computing. *Proc. ACM Softw.*

Eng. 1, FSE (2024), 767–788. https://doi.org/10.1145/3643761

[46] Yihao Hu, Amir Gilad, Kristin Stephens-Martinez, Sudeepa Roy, and Jun Yang. 2024. Qr-Hint: Actionable Hints Towards Correcting Wrong SQL Queries. *Proc. ACM Manag. Data* 2, 3 (2024), 164. https://doi.org/10.1145/3654995

[47] Yihao Hu, Zhengjie Miao, Zhiming Leong, Haechan Lim, Zachary Zheng, Sudeepa Roy, Kristin Stephens-Martinez, and Jun Yang. 2022. I-Rex: An Interactive Relational Query Debugger for SQL. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education V. 2*. 1180–1180.

[48] Jiansheng Huang, Ting Chen, AnHai Doan, and Jeffrey F Naughton. 2008. On the provenance of non-answers to queries over extracted data. *Proceedings of the VLDB Endowment* 1, 1 (2008), 736–747.

[49] Matteo Interlandi, Kshitij Shah, Sai Deep Tetali, Muhammad Ali Gulzar, Seunghyun Yoo, Miryung Kim, Todd Millstein, and Tyson Condie. 2015. Titian: Data provenance support in spark. In *Proceedings of the VLDB Endowment International Conference on Very Large Data Bases*, Vol. 9. NIH Public Access, 216.

[50] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing Source Code using a Neural Attention Model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 2073–2083. https://doi.org/10.18653/v1/P16-1195

[51] Hannu Jaakkola and Bernhard Thalheim. 2003. Visual SQL–high-quality ER-based query treatment. In *Conceptual Modeling for Novel Application Domains: ER 2003 Workshops ECOMO, IWCMQ, AOIS, and XSDM, Chicago, IL, USA, October 13, 2003. Proceedings 22*. Springer, 129–139.

[52] Grigoris Karvounarakis, Todd J Green, Zachary G Ives, and Val Tannen. 2013. Collaborative data sharing via update exchange and provenance. *ACM Transactions on Database Systems (TODS)* 38, 3 (2013), 1–42.

[53] Georgia Koutrika, Alkis Simitsis, and Yannis E Ioannidis. 2010. Explaining structured queries in natural language. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. IEEE, 333–344.

[54] Marie Le Guilly, Jean-Marc Petit, Vasile-Marian Scuturici, and Ihab F Ilyas. 2019. Explique: Interactive databases exploration with SQL. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 2877–2880.

[55] Seokki Lee, Bertram Ludäscher, and Boris Glavic. 2019. PUG: a framework and practical implementation for why and why-not provenance. *The VLDB Journal* 28, 1 (2019), 47–71.

[56] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763* (2024).

[57] Aristotelis Leventidis, Jiahui Zhang, Cody Dunne, Wolfgang Gatterbauer, HV Jagadish, and Mirek Riedewald. 2020. QueryVis: Logic-based diagrams help users understand complicated SQL queries faster. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2303–2318.

[58] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13067–13075.

[59] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).

[60] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.

[61] Zhengjie Miao, Tiangang Chen, Alexander Bendeck, Kevin Day, Sudeepa Roy, and Jun Yang. 2020. I-Rex: an interactive relational query explainer for SQL. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2997–3000.

[62] Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2019. Explaining Wrong Queries Using Small Examples. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 503–520. https://doi.org/10.1145/3299869.3319866

[63] Daphne Miedema and George Fletcher. 2021. SQLVis: Visual query representations for supporting SQL learners. In *2021 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 1–9.

[64] Tobias Müller, Benjamin Dietrich, and Torsten Grust. 2018. You Say 'What', I Hear 'Where' and 'Why'? (Mis-)Interpreting SQL to Derive Fine-Grained Provenance. *Proc. VLDB Endow.* 11, 11 (2018), 1536–1549. https://doi.org/10.14778/3236187.3236204

[65] Xing Niu, Boris Glavic, Ziyu Liu, Pengyuan Li, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, and Danica Porobic. 2021. Provenance-based Data Skipping. *Proc. VLDB Endow.* 15, 3 (2021), 451–464. https://doi.org/10.14778/3494124.3494130

[66] Laurel J. Orr, Srikanth Kandula, and Surajit Chaudhuri. 2019. Pushing Data-Induced Predicates Through Joins in Big-Data Clusters. *Proc. VLDB Endow.* 13, 3 (2019), 252–265. https://doi.org/10.14778/3368289.3368292

[67] PostgreSQL. [n.d.]. https://www.postgresql.org/.

[68] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. 2025. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=CvGqMD5OtX

[69] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in neural information processing systems* 36 (2023), 36339–36348.

[70] Kai Presler-Marshall, Sarah Heckman, and Kathryn T. Stolee. 2021. SQLRepair: Identifying and Repairing Mistakes in Student-Authored SQL Queries. In *43rd IEEE/ACM International Conference on Software Engineering: Software Engineering Education and Training, ICSE (SEET) 2021, Madrid, Spain, May 25-28, 2021*. IEEE, 199–210. https://doi.org/10.1109/ICSE-SEET52601.2021.00030

[71] Fotis Psallidas and Eugene Wu. 2018. Smoke: Fine-grained lineage at interactive speed. *arXiv preprint arXiv:1801.07237* (2018).

[72] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1*, Philip A. Bernstein (Ed.). ACM, 23–34. https://doi.org/10.1145/582095.582099

[73] Praveen Seshadri, Hamid Pirahesh, and T. Y. Cliff Leung. 1996. Complex Query Decorrelation. In *Proceedings of the Twelfth International Conference on Data Engineering, February 26 - March 1, 1996, New Orleans, Louisiana, USA*, Stanley Y. W. Su (Ed.). IEEE Computer Society, 450–458. https://doi.org/10.1109/ICDE.1996.492194

[74] Samuel Sanford Shapiro and Martin B Wilk. 1965. An analysis of variance test for normality (complete samples). *Biometrika* 52, 3-4 (1965), 591–611.

[75] Chang Shu, Yusen Zhang, Xiangyu Dong, Peng Shi, Tao Yu, and Rui Zhang. 2021. Logic-Consistency Text Generation from Semantic Parses. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, Online, 4414–4426. https://doi.org/10.18653/v1/2021.findings-acl.388

[76] Sivaprasad Sudhir, Wenbo Tao, Nikolay Pavlovich Laptev, Cyrille Habis, Michael J. Cafarella, and Samuel Madden. 2023. Pando: Enhanced Data Skipping with Logical Data Partitioning. *Proc. VLDB Endow.* 16, 9 (2023), 2316–2329. https://doi.org/10.14778/3598581.3598601

[77] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[78] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2021. Beyond Equi-joins: Ranking, Enumeration and Factorization. *Proc. VLDB Endow.* 14, 11 (2021), 2599–2612. https://doi.org/10.14778/3476249.3476306

[79] Margus Veanes, Nikolai Tillmann, and Jonathan de Halleux. 2010. Qex: Symbolic SQL Query Explorer. In *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25-May 1, 2010, Revised Selected Papers (Lecture Notes in Computer Science)*, Edmund M. Clarke and Andrei Voronkov (Eds.), Vol. 6355. Springer, 425–446. https://doi.org/10.1007/978-3-642-17511-4_24

[80] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *Proc. VLDB Endow.* 17, 11 (2024), 3602–3614. https://www.vldb.org/pvldb/vol17/p3602-wang.pdf

[81] Kun Xu, Lingfei Wu, Zhiguo Wang, Yansong Feng, and Vadim Sheinin. 2018. SQL-to-Text Generation with Graph-to-Sequence Model. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 931–936. https://doi.org/10.18653/v1/D18-1112

[82] Cong Yan, Yin Lin, and Yeye He. 2023. Predicate Pushdown for Data Science Pipelines. *Proc. ACM Manag. Data* 1, 2 (2023), 136:1–136:28. https://doi.org/10.1145/3589281

[83] Jing Ye, Yiwen Duan, Yonghong Yu, Victor Ma, Yang Gao, and Xing Chen. 2026. Beyond Text-to-SQL: Can LLMs Really Debug Enterprise ETL SQL? arXiv:2601.18119 [cs.AI] https://arxiv.org/abs/2601.18119

A ADDITIONAL DETAILS ON DEBUGGING PARADIGM

A.1 Set/Bag Operation Blocks

A SQL set/bag operation block has the form

|| Q_R UNION|INTERSECT|EXCEPT [ALL] Q_S

where Q_R and Q_S are subqueries. Let R and S denote the tables returned by Q_R and Q_S, respectively. The canonical execution procedure of this block consists of two stages: a sort stage that sorts both R and S by their row contents and a final output stage that merges them to produce the result table. To determine the order for the sort stage, we pick a particular ordering of all columns, preferring to reuse R's order (dictated by its IID) as much as possible. Further, to ensure a stable sort in the case of duplicates, we append each input table's IID to the column ordering as needed when sorting the table. The synthesized sort result IIDs have the same format. For instance, suppose that the IID for $R(A_1, A_2, A_3)$ is (A_3, A_1), which implies that R is free of duplicates. In this case, R is already sorted by $(A_3, A_1, A_2, \text{IID}(R)))$, and we would sort $S(B_1, B_2, B_3)$ accordingly by $(B_3, B_1, B_2, \text{IID}(S))$, assuming that S may contain duplicates. As another example, suppose that both input tables come from unsorted base tables with duplicates, and these tables' internal row ids serve as the IIDs. In this case, we would sort both tables by their columns, in order, followed by their respective IIDs.

The final output stage merges the two sorted tables. The output row IID always contains the value of columns in the same order as the sort stage IID, followed by a Boolean flag indicating the source input table (0 for R and 1 for S) and a sequence number (0-based) indicating its position among duplicates in the source input table. Depending on the set/bag operation involved, the stage's behavior and the last two components of the IID are defined differently. In the following, let t denote the content of a row in either R or S, and $R[t]$ and $S[r]$ the lists of duplicate rows in R and S (respectively) with this content.

- INTERSECT: We consider only the case when $R[t] \neq \emptyset$, and output only the first row of $R[t]$, with the last two components of the IID set to $(0, 0)$.
- EXCEPT: We consider only the case when $R[t] \neq \emptyset$. Only if $S[t] = \emptyset$, we output the first row of $R[t]$, with the last two components of the IID set to $(0, 0)$.
- UNION: If $R[t] \neq \emptyset$, we output the first row of $R[t]$, with the last two components of the IID set to $(0, 0)$. otherwise, we output the first row of $S[t]$, with the last two components of the IID set to $(1, 0)$.
- INTERSECT ALL: Let $m = \min\{|R[t]|, |S[t]|\}$. We output the first m rows of $R[t]$, with the last two components of the IID set to $(0, 0), \ldots, (0, m-1)$.
- EXCEPT ALL: Let $m = |R[t]| - |S[t]|$. Only if $m > 0$, we output the first m rows of $R[t]$, with the last two components of the IID set to $(0, 0), \ldots, (0, m-1)$.
- UNION ALL: Let $m = |R[t]| + |S[t]|$. We output all rows of $R[t]$ followed by all rows of $S[t]$, with the last two components of the IID set to $(0, 0), \ldots, (0, |R[t]| - 1)$ and then $(1, 0), \ldots, (1, |S[t] - 1|)$.

The input ("combo" would be somewhat a misnomer here) space for the debugging context is the concatenation of the rows of R followed by those of S. Hence, from the two input tables, only one row can be highlighted as currently active, and no input pinning is allowed because it is not useful in this context. Although the definitions above associate each output row with a particular input row, the I-Rex interface hides this detail, so users will still observe the SQL semantics. To be more specific, given an input row, we define its derivative "row" (if one exists) in the final output table as the group of all duplicate rows, which can only be pinned/unpinned together. The interface also provides an explanation for the number of duplicates or the absence of output rows for the given input row.

A.2 Join Expressions

I-Rex treats explicit JOIN expressions in FROM essentially as subqueries. Hence, each such expression gives rise to a separate debugging context with two input tables (base or derived). Inner joins can be handled in the same way as a more general SELECT block (Section 3.1). We focus on outer joins here. Let R and S denote the two input tables. We augment the input combo space by adding a special IID $\perp$ to each input table whose side of the output can be padded with NULLs. Intuitively, $\perp$ stands for "no row" from that input table, and behaves in the I-Rex interface as a special last row of the table. For example, for S in R LEFT JOIN S, we add $\perp$ to indicate "no row from S." If a row $r \in R$ joins with no row from S, the left outerjoin result should contain a row for r padded with NULLs in S columns; its IID would be $(r, \perp)$.

When ordering by IIDs, $\perp$ should be considered last in value. Internally, I-Rex represents $\perp$ in SQL using an IID whose components are all NULLs, assuming input rows do not have NULLs for all their IID components.[3] When generating queries involving comparison with IIDs possibly containing NULLs, however, we need to replace the ROW(...) comparisons with special code because SQL comparisons involving NULLs always yield the UNKNOWN truth value. For example, instead of generating ROW(A,B)>=ROW(1,2) when B can be NULL, we would generate

|| A>1 OR (A=1 AND (B IS NULL OR B>=2))

Selection of the active input combo and stepping work the same way as in SELECT (Section 3.1), except when the input combo involves a $\perp$, I-Rex show an explanation of why a NULL-padded result row is or is not produced, instead of the filter expression tree.

B ADDITIONAL DETAILS ON DEBUGGING PARADIGM

B.1 Proof of Theorem 4.1

Proof. First note that when $\Theta \Leftrightarrow \perp$ (i.e., Θ is equivalent to logical false), both queries are obviously equivalent as they return empty results on all possible database instances regardless of the evaluation of BLOOM_CHECK. Therefore, we now proceed to prove that both queries are equivalent when at least Q_1 returns a non-empty result (i.e., Θ is not equivalent to $\perp$).

Now assuming Q_1 and Q_2 are not equivalent, then there must exist input tables $\{R'_i\}$ where Q_1, Q_2 return results $\mathcal{V}_1$ and $\mathcal{V}_2$ respectively, and a tuple $e' \in \mathcal{V}_1$ but $e' \notin \mathcal{V}_2$ (it cannot happen vice versa as $\Theta \Leftarrow \Theta$ AND BLOOM_CHECK$(F, \langle\mathcal{A}\rangle)$), thus there must exist a subset $\mathcal{A}'$ from e' such that BLOOM_CHECK$(F, \langle e'[\mathcal{A}']\rangle)$ is evaluated to

[3] In the extremely rare case when this assumption is violated, we choose an unused value from each domain instead.

false. Observing that F is obtained from BLOOM_CHECK($\mathcal{V}_1[\mathcal{A}']$) and $e' \in \mathcal{V}_1$, this implies BLOOM_CHECK(F, $\langle e'[\mathcal{A}'] \rangle$) must be true as Bloom filter returns no false negative. Therefore, e' must also be in $\mathcal{V}_2$ and $\{R'_i\}$ does not exist. As a result, Q_1 and Q_2 must be equivalent. □

B.2 Optimizing Tracing and Pinning

With optimized page fetches, users can freely move around a table with a paginated display. However, a debugging context displays multiple tables, and I-Rex must coordinate the pages displayed across tables to show the end-to-end derivation from the current input combo to output, so that users can forward- or backward-trace (using pinning). We now discuss how to optimize these operations.

B.2.1 Forward Tracing. For forward tracing, we already have access to the input combo as well as their IIDs and row contents. Conceptually, to forward-trace through a particular stage, it suffices to determine the IID (say t) of the derivative row produced by this stage, if one exists. Then, given the target IID t, I-Rex searches the milestone table of the stage's result table for the page whose IID range contains t, and fetches this page. If the fetched page indeed contains t, we have successfully forward-traced through the stage; otherwise, we know that the input combo yields no result rows.

Determining the target IID in the first place is usually straightforward, thanks to the logical nature of our IIDs. For example, given the input combo $\langle s_2, f_1 \rangle$ in Figure 2, the target IID for the join & filter stage is simply the concatenation of the IIDs of s_2 and f_1. In cases where we cannot easily determine the target IID, I-Rex can compute it using a query. For example, to forward-trace through the group stage, we need to compute the GROUP BY expression value, the leading component of the result row IID. In this running example, the GROUP BY expression is simply s.bar, so we could have read its value Edge directly from row s_2. Otherwise, in general cases where the GROUP BY expression is complex (e.g., involving subqueries), I-Rex would generate a query to compute its value, e.g.:

```
|| SELECT s.bar -- arbitrary GROUP BY expression
|| FROM Serves s, Frequents f -- same as the original query block
|| WHERE (s.bar, s.beer) = ('Edge', 'Amstel') AND (f.bar, f.beer) = ('Ben',
||        'Edge'); -- use IID values to specify input combo
```

The cost of such queries is negligible because of the highly specific WHERE condition.

B.2.2 Pinning for Backward Tracing and Watchpointing. Without loss of generality, assume that a single derivative row is pinned.[4] Our goal is to determine the first (lexicographically) input combo in the pinned subspace. If only one input combo contributes to the pinned derivative row, we can simply infer the former from the pinned row's IID (the same applies when backward-tracing from a stage to its preceding stage). For example, in Figure 2, if a user pins the first member row of g_1 in the group table, the IID of this row, (Edge, (s_2, f_1)), will reveal the input combo $\langle s_2, f_1 \rangle$.

If multiple input combos contribute to one pinned derivative row, the situation is more complicated. Such cases arise when backward-tracing from a pinned row in a post-grouping stage, e.g., the final stage of Figure 2. Here, we first infer the group, identified by its GROUP BY expression value (say g), from the IID of the pinned row. There are two cases. First, if there are no additional pins on the input rows, we simply need to determine the first input combo contributing to group g. To this end, I-Rex searches the milestone table of the group table for the last page whose min_iid is less than $(g, -\infty)$, and fetches that page. If the fetched page contains any member row of g, the IID of the first such row will reveal the desired input combo. Otherwise, it can be shown that the next page's min_iid must give the desired input combo. Hence, recovering the input combo takes only one page fetch in the worst case.

Example B.1. Considering Figure 2 again. Assuming a page size of 3 (rows) for all tables and o_1 is pinned, I-Rex can quickly determine that the first page in the group table (which contains all tuples from g_0 and one tuple from g_1) is the last page whose min_iid is smaller than $\langle$Edge, $-\infty\rangle$. It thus generates the following query to fetch the first page of the group table:

```
|| SELECT (s.bar) -- group IID
||        ((s.bar, s.beer), (f.drinker, f.bar)) -- input combo IID
||        s.price * f.times -- sum_input
|| FROM Serves s, Frequents f
|| WHERE (...) -- original WHERE
|| AND ... -- sargable filtering
|| -- group IID and input combo IID filtering
|| AND ((s.bar), ((s.bar, s.beer), (f.drinker, f.bar))) >= (('Apex'), (('Apex',
||        'Corona'), ('Amy', 'Apex')))
|| AND ((s.bar), ((s.bar, s.beer), (f.drinker, f.bar))) < (('Edge'), (('Edge',
||        'Amstel'), ('Dan', 'Edge')))
|| ORDER BY 1, 2; -- first order by group, then by input combo
```

After obtaining the following result, it can be easily identified that the last tuple in the page carries the first input combo IIDs that contribute to o_1:

group IID	input combo IID	sum_input
(Apex)	((Apex, Corona), (Amy, Apex)))	1
(Apex)	((Apex, Dixie), (Amy, Apex)))	2
(Edge)	((Edge, Amstel), (Ben, Edge)))	16

Second, when there are additional pins on the input rows, I-Rex instead generates a query to determine the first input combo in the subspace further constrained by the additional pins. This query inherits the FROM and WHERE clauses from the original query, but further includes in WHERE conditions to restrict input rows by the pins and ensure that GROUP BY expression evaluates to g; it then computes the minimum input combo in SELECT.

Example B.2. Continuing from Theorem B.1. When s_2 is further pinned in addition to o_1, to compute the first input combo in the subspace of the group table, I-Rex generates the following query:

```
|| SELECT (s.bar) -- group IID
||        ((s.bar, s.beer), (f.drinker, f.bar)) -- input combo IID
|| FROM Serves s, Frequents f
|| WHERE (...) -- original WHERE
|| AND (s.bar, s.beer) = ('Edge', 'Amstel')
|| ORDER BY 1, 2 -- first order by group, then by input combo
|| LIMIT 1; -- only need first input combo
```

The above query locates $\langle s_2, f_1 \rangle$ as the first input combo, and I-Rex can now decide the page to fetch in the group table by a simple binary search using the group IID and input combo IID.

Recall that to support watchpointing, I-Rex automatically colors rows in each table relevant to the pinned subspace. The case where the pinned subspace contains only one input combo is straightforward: relevant rows are simply those in the input combo and those that are its derivatives. Otherwise, I-Rex extends the page-fetch query to return an additional column that indicates whether each row is relevant to the pinned subspace. The SELECT expression for

[4] Since I-Rex enforces that there is only one derivative row per stage, it suffices to consider the derivative row in the earliest stage.

this column is a Boolean expression testing whether the input rows conform to the pins and the GROUP BY expression evaluates to the same value as the pinned group.

B.3 System Implementation Details

Architecture. I-Rex has a client-server setup, with a Web frontend as the client and a middleware server between the client and the database server. In a common use case in education, many student users may run debugging sessions against one large, shared, read-only database. To ensure scalability and easy deployment, we adhere to a strictly stateless design, where neither the middleware nor the database server stores any session-specific information:

- When a user starts a debugging session, I-Rex middleware analyzes the query and decomposes it into a graph of query blocks in an internal representation. This representation specifies how each block is further broken down into stages, and contains metadata such as how to resolve external column references for correlated subqueries. The client caches this representation.
- When the user enters a debugging context (i.e., "calling" a query block), the client sends the cached representation of this block back to the middleware along with any parameter settings for the call. The middleware's *debugging context initializer* computes the milestone tables for all input tables of the debugging context and all result tables of its constituent stages, by querying the database (discussed below). The debugging context initializer also generates SQL query templates needed to support various debugging operations discussed earlier in this section. The client caches these milestone tables and SQL templates.
- When user carries out various operations in the debugging context, the client consults the cached milestone tables to instantiate required SQL queries according to the cached templates. It also caches the results of page-fetch queries, so repeated accesses to the same page, which can happen frequently during debugging, do not need recomputation. Cached pages can be evicted to make space for new ones.

Overall, note that debugging sessions leave no state on the middleware or the database. The middleware handles all complex SQL query analysis and rewrites, and the client simply needs to fill in the values of certain query parameters. The only cached data whose size depends on the database are the milestone tables. However, since there is only one milestone row per display page, and all columns are compact summaries of constant size, the milestone tables are orders of magnitude smaller than the corresponding tables. To make them even more scalable, milestones could be made hierarchical and refined on demand, but as shown later in Section 5, the current single-level design already works well.

Additional Details on Query Rewriting. As discussed earlier, all information needed for debugging can be computed by rewriting the original SQL query in some fashion. I-Rex performs most query rewriting in its debugging context initializer. We have already shown how to generate queries for fetching pages (Section 4.1) and supporting other debugging operations (Section B.2). As for computing milestones for each table in the debugging context, a single SQL query suffices, as illustrated by the following, which computes Table 1 (including the Bloom filters) for our running example:

```
|| WITH tmp(seq, iid, s_bar, s_beer, f_drinker, f_bar) AS (
||   SELECT ROW_NUMBER() OVER (ORDER BY s.bar,s.beer,f.drinker,f.bar) - 1, --
||          result row sequence number when sorted by IID
||     ((s.bar, s.beer), (f.drinker, f.bar)), -- IID
||     s.bar, s.beer, f.drinker, f.bar -- relevant for sargable & Bloom filtering
||   FROM Serves s, Frequents f WHERE (...) -- same as original query
|| )
|| SELECT MIN_IID(iid), -- per-page minimum IID
||   -- per-page range bounds for sargable fitering:
||   ARRAY[MIN(s_bar), MAX(s_bar)], ARRAY[MIN(s_beer), MAX(s_beer)],
||   ARRAY[MIN(f_drinker), MAX(f_drinker)], ARRAY[MIN(f_bar), MAX(f_bar)],
||   -- per-page Bloom filter:
||   BLOOM_GEN(p_partkey)
|| FROM tmp GROUP BY seq / page_size ORDER BY seq / page_size;
|| -- page_size is the number of rows per page
```

After generating the SQL queries as discussed above and earlier, I-Rex further applies several rewrite optimizations. First, if a scalar subquery contains no external column references, the debugging context initializer will simply precompute its result and replace the uses of this subqueries by its result. Second, for any sargable condition injected into a query, we consider pushing it down further into a subquery. Such cases often arise when, for example, we identify a range bound on some indexed column, and this column (or another column equated to it by WHERE) is referenced by a subquery as an external column; here, we inject the range bound into the subquery as well. Third, if the query's FROM contains a subquery or table defined by WITH, we check whether the injected conditions imply an equality or range condition the IID of the input table. If yes, we consult the milestones of the input table to construct sargable filters to further inject into the subquery defining the input table. We call this last optimization *recursive pushdown.* Finally, pushdown through outer joins, which is especially tricky, is discussed in Section B.

B.4 Pushdown through Outerjoins

Consider a page-fetch query that I-Rex generates for a full outer join debugging context, which has the form:

```
|| SELECT ROW(R.K, S.K) AS _iid
|| FROM R FULL JOIN S ON R.A = S.B
|| WHERE Θ(R, S);
```

Here, Θ is condition based on the IID range of the requested page, which constrains R and S. Efficient execution of this query requires pushing down filters inferred from Θ to R and S. Unfortunately, such pushdowns are not always safe through outer joins. For example, suppose that the requested page is the last one, and its IID range implies the filter on R to be $R.K$ IS NULL OR $R.K$>=100. Further suppose that some $s \in S$ joins with a single row $r \in R$ with $R.K$<100. The above page-fetch query should not return any output for s, since $\langle r, s\rangle$ does not belong to the requested page and hence fails the final WHERE. However, if we push the filter on R to below the outer join, the outer join will return a row containing s and R columns padded with NULLs, which passes the final WHERE.

To avoid the above issue and still enable pushdown, I-Rex first computes the inner join between R and S with pushdown. Then, only if the number of returned rows falls below the desired number on the requested page, which can be determined from the milestone table and should happen rarely, we issue additional queries to find rows in the outer join result but not in the inner.

C ADDITIONAL EXPERIMENTAL RESULTS

We present the remaining experiment results over TPC-H benchmark.

For each table in the TPC-H schema, we have the following indexes (all indexes are btree indexes in PostgreSQL):

- customer
  - primary index: c_custkey
  - secondary indexes: None
- lineitem
  - primary index: (l_orderkey, l_linenumber)
  - secondary indexes: l_partkey, l_suppkey, l_shipdate
- nation
  - primary index: n_nationkey
  - secondary indexes: n_name, n_regionkey
- orders
  - primary index: o_orderkey
  - secondary indexes: o_custkey, o_orderdate
- part
  - primary index: p_partkey
  - secondary indexes: None
- partsupp
  - primary index: (ps_partkey, ps_suppkey)
  - secondary indexes: ps_suppkey
- region
  - primary index: r_regionkey
  - secondary indexes: None
- supplier
  - primary index: s_suppkey
  - secondary indexes: s_name, s_phone

We ran experiments for three different page size: 50, 100 and 200 for all tables (stages) in all TPC-H queries. For each table, we prepared three queries: milestone query, page query and table query and thus collecting the following data over all testing instances (i.e., 1GB, 5GB and 10GB):

- The execution time and output size of the milstone query and the table query.
- The execution time of the page query and baseline query (by rewritting the table query with `OFFSET` and `LIMIT`) for retrieving the first page ("head"), middle page ("mid") and the second last ("tail") page.

Since each query potentially contains multiple query blocks and subsequently multiple tables, we present the statistics for the largest table (measured in MB) to compute for each query. For page size 50, 100 and 200, the experiment results are shown in Table 5, Table 6 and Table 7 respectively.

In summary, we make the following conclusions:

- The milestone queries can run slower than the table queries, but with acceptable delays since they are run at the beginning of the debugging session without affecting debugging operations later. In some cases, the milestone queries run faster than the table queries. On the other hand, the output sizes of the milestone queries are almost always smaller than those of the table queries by a rough factor of the page size.
- The optimization for page query almost always outperforms the baseline, and differences between the execution time grow as the database size grows, especially for "mid" and "tail" pages. The optimizations are sensitive to page size but insensitive to the database size. There are only few cases where the optimizations "over-hint" the PosgreSQL optimizer and cause the execution time to be roughly the same as the baseline.

		1GB		5GB		10GB	
Query	Page	Opt.	Base	Opt.	Base	Opt.	Base
Q1	head	**0.053**	0.067	**0.058**	0.106	0.071	**0.059**
	mid	**0.038**	276.977	**0.123**	10719.953	**0.168**	21826.43
	tail	**0.034**	553.887	**0.033**	21439.8	**0.052**	43652.801
Q2	head	**5.758**	251.032	**6.356**	1707.231	**6.954**	3291.403
	mid	**7.236**	253.723	**5.953**	1716.116	**8.916**	3571.981
	tail	**6.596**	256.414	**6.832**	1712.497	**6.503**	3852.56
Q3	head	12.57	**4.225**	**81.789**	407.539	**59.029**	590.042
	mid	**6.165**	318.842	**145.043**	4937.49	**58.346**	22685.161
	tail	**7.179**	633.459	**143.534**	9467.44	**34.891**	44780.279
Q4	head	0.445	**0.364**	0.405	**0.362**	3.39	**2.122**
	mid	**0.445**	108.699	**1.318**	554.128	**5.365**	8912.874
	tail	**0.383**	217.034	**2.006**	1107.893	**3.216**	17823.626
Q5	head	8.909	**8.657**	**9.142**	13.343	420.739	**314.969**
	mid	**7.653**	1287.24	**23.182**	10036.965	**193.466**	28344.422
	tail	**5.98**	2508.011	**7.776**	20060.586	**193.096**	56373.876
Q6	head	0.189	**0.156**	0.175	**0.154**	0.405	**0.156**
	mid	**0.213**	216.07	**0.202**	1168.038	**3.059**	16448.599
	tail	**0.147**	431.984	**0.197**	2335.922	**1.594**	32897.042
Q7	head	20.642	**14.371**	68.84	**14.918**	985.063	**71.079**
	mid	**12.155**	395.358	**41.39**	3065.17	**851.135**	15909.36
	tail	**14.622**	776.345	**41.266**	6105.881	**907.042**	27296.999
Q8	head	**4.148**	4.407	**4.7**	35.101	317.395	**45.033**
	mid	**4.32**	1656.216	**28.169**	6557.276	**238.396**	20430.873
	tail	**3.984**	2970.844	**4.587**	13079.451	**245.715**	40816.712
Q9	head	**0.853**	242.203	**143.601**	12191.21	**51.197**	85054.771
	mid	**1.245**	1008.268	**199.13**	14899.526	**38.146**	69401.355
	tail	**0.765**	1774.333	**131.757**	17607.842	**35.69**	54535.253
Q10	head	0.939	**0.748**	62.441	**1.509**	19.73	**0.912**
	mid	**0.835**	800.388	**28.382**	15660.853	**76.849**	20184.374
	tail	**0.772**	1600.028	**83.471**	18930.806	**8.615**	40367.836
Q11	head	23.582	**1.898**	134.269	**6.779**	262.009	**7.036**
	mid	**22.03**	32.884	**169.394**	221.772	**244.886**	393.363
	tail	**17.811**	63.87	**130.579**	436.765	**233.001**	779.69
Q12	head	0.858	**0.615**	9.142	**0.569**	**1.741**	2.327
	mid	**0.808**	285.367	**0.558**	7589.805	**0.711**	17582.209
	tail	**0.866**	570.118	**0.619**	12918.587	**0.549**	35162.092
Q13	head	0.244	**0.197**	0.184	**0.172**	0.193	**0.186**
	mid	**0.195**	470.197	**0.191**	2562.524	**0.182**	5584.982
	tail	**0.172**	940.197	**0.163**	5124.876	**0.186**	11169.778
Q14	head	4.875	**1.2**	33.992	**3.162**	7.135	**1.846**
	mid	**4.455**	116.905	**32.801**	717.536	**7.643**	1573.678
	tail	**4.392**	232.611	**30.97**	1431.91	**5.695**	3145.51
Q15	head	0.238	**0.221**	0.247	**0.234**	1.315	**0.276**
	mid	**0.241**	156.641	**0.182**	922.131	**12.682**	1779.888
	tail	**0.163**	313.061	**6.841**	1844.028	**2.764**	3559.5
Q16	head	1.682	**1.39**	6.656	**6.074**	12.276	**11.936**
	mid	**1.438**	140.749	**6.191**	844.365	**13.452**	1700.343
	tail	**1.486**	280.107	**5.777**	1682.656	**12.02**	3388.75
Q17	head	2.538	**2.236**	15.809	**2.056**	**20.733**	125.812
	mid	**2.286**	1536.001	**5.141**	9945.394	**5.923**	25862.823
	tail	**2.409**	3069.766	**4.89**	19888.732	**4.945**	51599.835
Q18	head	**2532.81**	5870.729	**14184.028**	46234.889	**37419.067**	120576.875
	mid	**2456.265**	7991.925	**16215.405**	55316.965	**32434.602**	133646.155
	tail	**2787.514**	10113.122	**13983.933**	64399.041	**33393.882**	146715.434
Q19	head	**25.461**	34.29	178.18	**51.645**	1011.015	**60.365**
	mid	**25.324**	76.065	**129.633**	339.639	**138.908**	589.525
	tail	**24.978**	117.84	**117.056**	627.633	**59.341**	1118.685
Q20	head	0.094	**0.053**	0.086	**0.047**	**0.184**	0.659
	mid	**0.077**	49.901	**1.133**	661.308	**2.122**	1564.677
	tail	**0.086**	99.748	**30.041**	1322.57	**0.842**	3128.694
Q21	head	**16.769**	823.461	**189.848**	15873.412	**1039.691**	46780.254
	mid	**11.981**	830.494	**232.896**	15499.488	**909.955**	47334.357
	tail	**13.678**	837.527	**284.158**	15125.565	**1099.474**	46780.254
Q22	head	**0.608**	31.927	**0.838**	159.24	**1.707**	318.704
	mid	**0.673**	52.952	**0.82**	261.309	**2.324**	529.83
	tail	**0.512**	73.61	**0.869**	363.378	**1.838**	740.955

(a) Optimization vs. Baseline for Page Query Execution Time

		Time (ms)		Output Size (MB)	
Query	DB size	Milestone	Table	Milestone	Table
Q1	1	8331.914	**1388.336**	**31.159**	761.653
	5	44032.912	**23151.685**	**155.853**	3809.747
	10	87464.164	**46420.358**	**311.667**	7618.516
Q2	1	**215.464**	329.321	**0.006**	0.129
	5	**1356.54**	2386.544	**0.031**	0.647
	10	**2698.862**	4956.342	**0.062**	1.31
Q3	1	**1044.567**	1645.413	**0.216**	9.358
	5	**5933.313**	22270.548	**1.056**	45.735
	10	**13848.889**	43462.104	**2.145**	92.932
Q4	1	377.848	**347.824**	**0.382**	6.367
	5	1640.64	**1507.237**	**1.899**	31.649
	10	**11874.966**	25455.856	**3.79**	63.165
Q5	1	**2142.641**	2747.291	**0.936**	30.979
	5	**11543.048**	20464.069	**4.675**	154.836
	10	**27796.453**	58846.691	**9.325**	308.884
Q6	1	763.76	**595.106**	**1.795**	43.884
	5	3899.765	**2675.542**	**8.951**	218.804
	10	**9705.647**	34310.126	**17.923**	438.125
Q7	1	**810.777**	987.756	**0.152**	0.947
	5	**6335.476**	6637.603	**0.737**	4.602
	10	**14646.955**	35110.268	**1.489**	9.306
Q8	1	4960.836	**3186.478**	**11.67**	65.639
	5	23772.766	**17879.496**	**58.054**	326.55
	10	50267.721	**41981.095**	**116.284**	654.091
Q9	1	**2227.58**	2517.948	**4.267**	23.998
	5	**17141.161**	19767.02	**20.989**	118.061
	10	**31764.758**	53459.127	**42.012**	236.312
Q10	1	**1583.518**	1695.667	**2.897**	77.861
	5	9485.609	**9060.029**	**14.538**	390.698
	10	**20332.249**	49173.573	**29.047**	780.631
Q11	1	126.067	**82.778**	**0.243**	5.928
	5	**538.645**	590.908	**1.152**	28.146
	10	**985.077**	18047.688	**2.299**	56.193
Q12	1	**1108.119**	1502.238	**0.788**	10.507
	5	**5109.859**	16516.219	**3.947**	52.627
	10	**10723.043**	43430.112	**7.879**	105.057
Q13	1	4453.644	**1820.081**	**7.192**	74.4
	5	22620.462	**10634.663**	**35.96**	372.001
	10	46633.065	**22689.628**	**71.92**	744.001
Q14	1	**267.11**	344.042	**0.539**	18.571
	5	**1537.443**	5938.491	**2.694**	92.793
	10	**3053.81**	43108.788	**5.392**	185.705
Q15	1	697.134	**440.953**	**2.165**	52.914
	5	**3615.734**	6797.217	**10.845**	265.099
	10	**7422.926**	40451.245	**21.696**	530.346
Q16	1	382.706	**300.081**	**0.741**	16.455
	5	**1855.771**	4245.039	**3.678**	81.74
	10	**3654.478**	4303.77	**7.378**	163.946
Q17	1	6962.627	**6846.472**	**1.44**	11.2
	5	**18483.761**	22136.792	**7.2**	56.0
	10	**38778.384**	59549.219	**14.4**	112.0
Q18	1	15984.578	**12086.886**	**52.215**	1272.737
	5	95566.664	**87403.308**	**260.86**	6358.46
	10	**174787.04**	181889.789	**521.399**	12709.096
Q19	1	**145.722**	244.134	**0.044**	1.501
	5	**770.808**	15055.31	**0.217**	7.47
	10	**1270.167**	68809.099	**0.421**	14.506
Q20	1	2114.795	**1503.764**	**6.577**	1056.214
	5	**10328.037**	25180.76	**32.885**	5279.964
	10	**22092.932**	60763.015	**65.691**	10557.545
Q21	1	**869.86**	1039.671	**0.051**	1.332
	5	**10193.128**	37567.462	**0.25**	6.538
	10	**25156.809**	102244.18	**0.511**	13.417
Q22	1	120.044	**77.932**	**0.046**	0.405
	5	**472.207**	497.835	**0.229**	2.035
	10	**897.368**	2021.357	**0.458**	4.069

(b) Milstone Query vs. Table Query

Table 5: Experiment results for Page Size 50. Shorter time and smaller sizes are boldfaced.

		1GB		5GB		10GB	
Query	Page	Opt.	Base	Opt.	Base	Opt.	Base
Q1	head	**0.071**	0.073	0.079	**0.069**	0.084	**0.069**
	mid	**0.055**	296.325	**0.069**	9693.89	**2.16**	22897.01
	tail	**0.05**	592.576	**0.055**	19387.711	**0.094**	45793.951
Q2	head	**13.061**	295.438	**15.513**	1734.351	**14.555**	3311.46
	mid	**12.952**	275.061	**11.556**	1751.559	**14.029**	3591.94
	tail	**11.611**	254.684	**17.868**	1738.402	**13.057**	3872.421
Q3	head	**2.198**	7.366	**62.352**	63.668	**40.817**	485.093
	mid	**1.76**	323.121	**76.662**	4827.628	**47.888**	20987.232
	tail	**1.762**	638.876	**24.29**	9591.589	**35.286**	41449.976
Q4	head	0.643	**0.631**	0.656	**0.652**	31.234	**3.163**
	mid	**0.683**	110.39	**0.753**	573.421	**8.977**	9141.04
	tail	**0.665**	220.149	**12.071**	1146.19	**4.969**	18278.917
Q5	head	14.197	**13.368**	45.254	**17.582**	683.39	**473.463**
	mid	**13.586**	1244.566	**43.844**	10020.093	**343.844**	29541.882
	tail	**10.241**	2475.765	**14.55**	19971.901	**300.573**	57266.427
Q6	head	0.385	**0.32**	0.399	**0.333**	1.518	**0.319**
	mid	**0.363**	218.425	**0.349**	1196.178	**2.845**	16829.839
	tail	**0.294**	435.856	**0.384**	2392.023	**4.812**	33659.359
Q7	head	**26.172**	26.356	96.368	**34.798**	1772.658	**46.352**
	mid	**20.338**	387.671	**36.935**	3093.804	**1073.085**	16393.14
	tail	**22.026**	748.985	**32.845**	6078.97	**948.516**	32739.927
Q8	head	9.067	**8.502**	**8.987**	48.048	509.672	**34.627**
	mid	**6.755**	1695.254	**29.111**	6617.096	**422.185**	20162.913
	tail	**7.429**	2995.663	**10.025**	13186.143	**364.929**	40291.199
Q9	head	**1.477**	231.532	**67.733**	8417.397	**94.242**	74380.749
	mid	**1.375**	962.979	**77.736**	12923.744	**61.954**	64993.676
	tail	**1.867**	1694.426	**53.516**	17430.09	**66.841**	58048.861
Q10	head	1.46	**1.245**	21.716	**2.257**	26.641	**2.529**
	mid	**1.201**	797.21	**15.774**	8107.626	**22.302**	21555.761
	tail	**1.364**	1593.175	**21.038**	13905.496	**13.519**	43108.993
Q11	head	36.541	**3.3**	248.452	**8.555**	448.46	**12.203**
	mid	38.196	**36.848**	**219.147**	222.082	513.017	**390.707**
	tail	**38.557**	70.396	**202.447**	435.609	**397.68**	769.212
Q12	head	1.105	**1.08**	11.862	**1.222**	2.863	**1.216**
	mid	**0.976**	296.293	**1.204**	7060.195	**1.321**	17357.688
	tail	**1.126**	591.506	**1.017**	12045.238	**0.893**	34714.16
Q13	head	0.347	**0.346**	**0.305**	0.309	0.341	**0.331**
	mid	**0.331**	458.786	**0.291**	2603.571	**0.285**	5598.169
	tail	**0.331**	917.225	**0.26**	5206.832	**0.308**	11196.008
Q14	head	5.512	**2.71**	35.8	**5.729**	78.23	**14.473**
	mid	**4.76**	118.153	**35.366**	718.17	**76.052**	1600.618
	tail	**4.772**	233.595	**35.272**	1430.611	**79.114**	3186.763
Q15	head	0.353	**0.297**	**0.328**	0.329	4.253	**0.35**
	mid	**0.371**	157.853	**2.625**	930.38	**12.812**	1785.893
	tail	**0.392**	315.409	**0.707**	1860.43	**4.657**	3571.436
Q16	head	2.137	**1.756**	6.243	**5.975**	**12.268**	14.385
	mid	**1.931**	143.953	**6.294**	854.68	**14.295**	1753.734
	tail	**1.773**	286.15	**6.041**	1703.385	**13.032**	3493.083
Q17	head	4.729	**3.941**	9.248	**4.222**	32.706	**3.904**
	mid	**4.541**	1527.395	**11.298**	9669.438	**13.855**	27819.374
	tail	**4.505**	3050.85	**12.892**	19334.653	**20.307**	55634.844
Q18	head	**2527.82**	5984.869	**15935.661**	45765.329	**34808.126**	126117.683
	mid	**2618.646**	8093.631	**16087.319**	55018.855	**34021.36**	139702.667
	tail	**2663.236**	10202.394	**17309.887**	64272.382	**37965.734**	153287.651
Q19	head	**43.729**	68.104	197.71	**105.373**	**261.355**	1188.735
	mid	**42.12**	94.035	**144.187**	370.888	**375.334**	1146.398
	tail	**44.463**	119.965	**151.102**	630.924	**209.468**	1136.618
Q20	head	0.158	**0.095**	0.173	**0.084**	**0.291**	1.433
	mid	**0.178**	50.034	**0.116**	352.756	**2.826**	1696.115
	tail	**0.148**	99.973	**1.263**	705.428	**1.025**	3390.798
Q21	head	**31.819**	874.487	**2616.526**	13686.317	**1375.21**	52290.601
	mid	**21.822**	852.312	**1529.724**	13691.07	**2528.902**	51884.652
	tail	**22.996**	835.458	**668.835**	13686.317	**2307.124**	51478.704
Q22	head	**0.994**	33.647	**0.967**	159.614	**4.113**	322.177
	mid	**1.021**	52.94	**1.392**	267.151	**2.617**	534.946
	tail	**1.265**	72.233	**1.525**	374.689	**2.231**	747.714

(a) Optimization vs. Baseline for Page Query Execution Time

		Time (ms)		Output Size (MB)	
Query	DB size	Milestone	Table	Milestone	Table
Q1	1	8308.278	**1350.574**	**15.579**	761.653
	5	43646.179	**21979.09**	**77.927**	3809.747
	10	87349.051	**47552.632**	**155.834**	7618.516
Q2	1	**204.105**	303.443	**0.003**	0.129
	5	**1419.539**	2199.755	**0.015**	0.647
	10	**2718.322**	5082.711	**0.031**	1.31
Q3	1	**1049.375**	1435.624	**0.108**	9.358
	5	**6184.207**	15819.558	**0.528**	45.735
	10	**14013.955**	42987.942	**1.072**	92.932
Q4	1	**322.772**	368.223	**0.191**	6.367
	5	1596.769	**1593.545**	**0.95**	31.649
	10	**11752.173**	25611.759	**1.895**	63.165
Q5	1	**2148.69**	2753.883	**0.468**	30.979
	5	**11514.031**	20612.307	**2.337**	154.836
	10	**27627.926**	58496.928	**4.662**	308.884
Q6	1	789.488	**570.995**	**0.898**	43.884
	5	3902.241	**2784.104**	**4.476**	218.804
	10	**9796.535**	33640.745	**8.962**	438.125
Q7	1	**826.424**	968.716	**0.076**	0.947
	5	**6563.466**	6701.097	**0.369**	4.602
	10	**14712.534**	31113.968	**0.745**	9.306
Q8	1	5090.232	**3210.232**	**5.835**	65.639
	5	23697.023	**17788.411**	**29.027**	326.55
	10	50386.193	**43219.432**	**58.142**	654.091
Q9	1	**2236.322**	2488.499	**2.134**	23.998
	5	**17082.537**	19866.147	**10.495**	118.061
	10	**32766.722**	53930.664	**21.006**	236.312
Q10	1	**1674.93**	1711.13	**1.449**	77.861
	5	9465.303	**8851.235**	**7.269**	390.698
	10	**20179.256**	52479.808	**14.524**	780.631
Q11	1	123.001	**84.227**	**0.121**	5.928
	5	548.9	**523.548**	**0.576**	28.146
	10	**994.286**	18564.246	**1.149**	56.193
Q12	1	**1128.692**	1247.274	**0.394**	10.507
	5	**5149.191**	15537.438	**1.974**	52.627
	10	**10833.248**	45744.079	**3.94**	105.057
Q13	1	4595.15	**2009.385**	**3.596**	74.4
	5	22778.525	**10871.238**	**17.98**	372.001
	10	46638.458	**22716.047**	**35.96**	744.001
Q14	1	**292.902**	349.172	**0.27**	18.571
	5	**1530.04**	5593.783	**1.347**	92.793
	10	**3086.853**	48864.272	**2.696**	185.705
Q15	1	738.978	**442.769**	**1.083**	52.914
	5	**3648.849**	7119.87	**5.423**	265.099
	10	**7489.047**	45756.265	**10.848**	530.346
Q16	1	383.589	**301.347**	**0.37**	16.455
	5	**1787.028**	2912.128	**1.839**	81.74
	10	**3568.993**	4667.728	**3.689**	163.946
Q17	1	6788.831	**6659.41**	**0.72**	11.2
	5	17970.305	**17523.58**	**3.6**	56.0
	10	**39062.359**	69424.705	**7.2**	112.0
Q18	1	15496.731	**12175.682**	**26.108**	1272.737
	5	**80673.913**	85605.048	**130.43**	6358.46
	10	**172242.749**	188263.583	**260.7**	12709.096
Q19	1	**128.724**	253.978	**0.022**	1.501
	5	**770.826**	4591.412	**0.109**	7.47
	10	**1244.035**	73082.065	**0.211**	14.506
Q20	1	2058.655	**1527.349**	**3.289**	1056.214
	5	**10373.605**	26763.565	**16.443**	5279.964
	10	**22292.044**	67862.73	**32.845**	10557.545
Q21	1	**879.998**	1045.606	**0.026**	1.332
	5	**10048.959**	31560.391	**0.125**	6.538
	10	**25372.791**	115913.909	**0.256**	13.417
Q22	1	118.981	**78.522**	**0.023**	0.405
	5	**538.636**	550.1	**0.114**	2.035
	10	**904.977**	1966.22	**0.229**	4.069

(b) Milstone Query vs. Table Query

Table 6: Experiment results for Page Size 100. Shorter time and smaller sizes are boldfaced.

		1GB		5GB		10GB	
Query	Page	Opt.	Base	Opt.	Base	Opt.	Base
Q1	head	**0.096**	0.106	0.128	**0.113**	0.211	**0.106**
	mid	**0.081**	277.353	**0.149**	9062.271	**1.296**	21098.182
	tail	**0.08**	554.601	**0.131**	18124.429	**0.132**	42196.259
Q2	head	**14.495**	258.075	**27.029**	1721.245	**26.565**	3314.087
	mid	**15.403**	257.304	**33.257**	1723.475	**27.33**	3599.783
	tail	**14.901**	256.534	**34.94**	1721.245	**27.07**	3885.479
Q3	head	**3.846**	12.189	**113.863**	264.195	**105.038**	626.635
	mid	**3.635**	311.993	**64.68**	4951.953	**122.484**	21306.666
	tail	**3.79**	611.796	**61.986**	9560.932	**99.174**	40314.53
Q4	head	1.661	**1.226**	**1.483**	1.534	31.712	**7.704**
	mid	**1.355**	110.832	**1.57**	567.687	**13.946**	8836.858
	tail	**1.319**	220.437	**2.908**	1133.84	**10.08**	17666.011
Q5	head	**23.257**	26.016	**32.109**	40.857	1097.932	**735.291**
	mid	**21.175**	1265.583	**33.95**	9974.61	**648.512**	28437.192
	tail	**18.287**	2505.151	**26.898**	19908.362	**543.151**	55813.192
Q6	head	**0.791**	1.039	0.764	**0.648**	2.651	**0.774**
	mid	**0.747**	220.038	**0.719**	1200.926	**4.505**	16894.755
	tail	**0.677**	437.797	**0.888**	2401.204	**2.423**	33589.6
Q7	head	**44.788**	47.835	122.091	**61.945**	2770.886	**68.498**
	mid	**44.945**	420.81	**71.516**	3093.383	**1552.064**	19057.859
	tail	**44.876**	775.126	**59.541**	6123.132	**1437.85**	30266.428
Q8	head	17.249	**14.642**	**25.396**	43.739	664.931	**30.171**
	mid	**14.123**	1646.464	**30.191**	6700.285	**600.04**	22275.784
	tail	**14.213**	2969.118	**18.549**	13356.832	**596.333**	44521.396
Q9	head	**2.317**	226.431	**98.203**	7932.852	**94.028**	62387.44
	mid	**2.26**	984.02	**99.992**	12663.04	**94.809**	58133.264
	tail	**2.04**	1741.609	**95.246**	17393.229	**93.872**	55136.418
Q10	head	2.702	**2.553**	42.657	**3.849**	51.023	**3.883**
	mid	**2.37**	790.804	**29.605**	7651.506	**31.641**	21214.82
	tail	**2.596**	1579.054	**30.024**	13908.715	**32.623**	42425.757
Q11	head	70.328	**6.789**	377.987	**12.112**	781.963	**16.625**
	mid	70.67	**35.681**	391.113	**224.163**	841.303	**394.805**
	tail	74.526	**64.572**	**369.919**	436.215	**750.659**	772.984
Q12	head	3.016	**1.921**	13.035	**2.318**	4.646	**3.263**
	mid	**2.906**	284.822	**2.219**	7395.704	**3.03**	18224.874
	tail	**1.961**	567.722	**1.727**	12139.442	**2.067**	36446.485
Q13	head	**0.577**	0.583	0.867	**0.495**	0.59	**0.526**
	mid	**0.621**	462.139	**0.765**	2654.764	**0.553**	5586.485
	tail	**0.541**	923.696	**0.947**	6597.823	**0.582**	11172.445
Q14	head	5.956	**3.442**	38.885	**22.228**	99.691	**18.986**
	mid	**5.508**	116.254	**37.352**	741.654	**97.37**	1605.492
	tail	**5.345**	229.065	**35.672**	1461.079	**97.398**	3191.998
Q15	head	0.618	**0.526**	0.594	**0.507**	10.788	**0.67**
	mid	**0.542**	157.453	**9.813**	946.187	**9.923**	1816.009
	tail	**0.62**	314.379	**2.305**	1891.866	**12.64**	3631.348
Q16	head	2.014	**1.749**	**6.852**	9.216	12.87	**12.089**
	mid	**2.402**	138.499	**7.195**	868.328	**14.227**	1774.517
	tail	**2.364**	275.248	**6.7**	1727.44	**13.133**	3536.946
Q17	head	8.908	**7.311**	25.304	**6.955**	48.554	**8.495**
	mid	**8.768**	1493.372	**21.24**	8114.816	**29.671**	27939.016
	tail	**8.532**	2979.433	**20.793**	16222.677	**29.471**	55869.537
Q18	head	**2620.295**	5958.982	**19362.324**	55544.906	**36723.62**	124199.714
	mid	**2629.855**	8101.114	**18941.112**	63877.35	**37481.778**	139121.515
	tail	**2625.798**	10243.246	**19651.261**	72209.794	**38882.822**	154043.316
Q19	head	**53.896**	116.645	**249.324**	937.218	**358.445**	1142.017
	mid	**57.568**	116.792	**241.046**	789.953	**536.344**	1126.027
	tail	**58.638**	116.939	**127.755**	642.689	**241.034**	1142.017
Q20	head	0.277	**0.111**	0.271	**0.094**	**1.557**	3.551
	mid	**0.247**	49.705	**0.239**	410.093	**3.099**	1619.594
	tail	**0.257**	99.299	**0.301**	820.092	**1.567**	3235.638
Q21	head	**62.057**	822.659	**665.833**	15239.98	**4164.863**	51430.138
	mid	**42.706**	823.801	**368.077**	15062.294	**4097.071**	51530.172
	tail	**45.318**	824.942	**204.653**	14884.608	**3898.247**	51430.138
Q22	head	**1.883**	34.867	**1.865**	157.814	**3.948**	319.443
	mid	**1.962**	53.499	**2.578**	263.99	**3.639**	532.957
	tail	**1.787**	72.131	**1.9**	368.158	**3.279**	746.472

(a) Optimization vs. Baseline for Page Query Execution Time

		Time (ms)		Output Size (MB)	
Query	DB size	Milestone	Table	Milestone	Table
Q1	1	8266.691	**1421.319**	**7.79**	761.653
	5	43465.288	**19863.953**	**38.964**	3809.747
	10	87255.985	**44610.896**	**77.917**	7618.516
Q2	1	**211.127**	316.426	**0.003**	0.129
	5	**1368.572**	2102.53	**0.008**	0.647
	10	**2731.286**	4999.87	**0.016**	1.31
Q3	1	**1076.937**	1434.405	**0.054**	9.358
	5	**6026.493**	17564.81	**0.264**	45.735
	10	**14199.75**	39549.761	**0.536**	92.932
Q4	1	**322.347**	347.205	**0.096**	6.367
	5	1572.678	**1554.638**	**0.475**	31.649
	10	**11860.375**	25225.163	**0.948**	63.165
Q5	1	**2166.316**	2745.94	**0.234**	30.979
	5	**11554.084**	20418.954	**1.169**	154.836
	10	**28022.606**	58548.771	**2.332**	308.884
Q6	1	757.998	**583.419**	**0.449**	43.884
	5	3864.204	**2686.491**	**2.238**	218.804
	10	**9770.35**	34763.653	**4.481**	438.125
Q7	1	**842.105**	1012.523	**0.038**	0.947
	5	**6398.453**	6636.314	**0.184**	4.602
	10	**15016.243**	35809.983	**0.372**	9.306
Q8	1	4944.503	**3274.571**	**2.918**	65.639
	5	23695.185	**17903.258**	**14.514**	326.55
	10	50859.253	**44384.607**	**29.071**	654.091
Q9	1	**2152.975**	2460.966	**1.067**	23.998
	5	**17039.021**	19611.865	**5.247**	118.061
	10	**32557.873**	54465.756	**10.503**	236.312
Q10	1	**1574.498**	1681.446	**0.724**	77.861
	5	9602.298	**8902.297**	**3.635**	390.698
	10	**20372.195**	50133.868	**7.262**	780.631
Q11	1	117.053	**80.026**	**0.061**	5.928
	5	552.067	**541.54**	**0.288**	28.146
	10	**998.988**	18808.77	**0.575**	56.193
Q12	1	**1129.749**	1261.052	**0.197**	10.507
	5	**5083.999**	14841.59	**0.987**	52.627
	10	**10966.114**	41976.288	**1.97**	105.057
Q13	1	4588.313	**1920.396**	**1.798**	74.4
	5	29358.318	**12564.698**	**8.59**	372.001
	10	46773.33	**22870.237**	**17.98**	744.001
Q14	1	**285.983**	330.957	**0.135**	18.571
	5	**1564.669**	16337.804	**0.674**	92.793
	10	**3066.596**	42790.62	**1.348**	185.705
Q15	1	671.876	**427.38**	**0.541**	52.914
	5	**3607.563**	15686.577	**2.712**	265.099
	10	**7351.63**	44726.829	**5.424**	530.346
Q16	1	358.35	**301.054**	**0.185**	16.455
	5	**1764.462**	2179.797	**0.92**	81.74
	10	**3583.625**	4496.268	**1.845**	163.946
Q17	1	6743.266	**6485.26**	**0.36**	11.2
	5	**19005.803**	21509.807	**1.8**	56.0
	10	**39388.936**	64307.077	**3.6**	112.0
Q18	1	15197.81	**12058.411**	**13.054**	1272.737
	5	**81764.098**	87866.796	**65.215**	6358.46
	10	**170623.134**	188950.673	**130.35**	12709.096
Q19	1	**123.344**	260.007	**0.011**	1.501
	5	**751.385**	24049.184	**0.054**	7.47
	10	**1224.462**	81688.742	**0.105**	14.506
Q20	1	2040.119	**1499.107**	**1.644**	1056.214
	5	**10370.233**	24150.297	**8.221**	5279.964
	10	**22286.322**	67750.294	**16.423**	10557.545
Q21	1	**904.569**	1049.034	**0.013**	1.332
	5	**10135.259**	23102.873	**0.063**	6.538
	10	**25376.696**	115486.433	**0.128**	13.417
Q22	1	110.451	**79.542**	**0.012**	0.405
	5	**476.884**	494.633	**0.057**	2.035
	10	**928.369**	1868.3	**0.114**	4.069

(b) Milstone Query vs. Table Query

Table 7: Experiment results for Page Size 200. Shorter time and smaller sizes are boldfaced.

D USER STUDY APPENDIX

D.1 Debugging Questions in Quiz

The user study is conducted with an instance of beer database with the following schema (keys are underlined):

- Drinker (name, address)
- Bar (name, address)
- Beer (name, brewery)
- Frequents (drinker, bar, times_a_week)
- Serves (bar, beer, price)
- Likes (drinker, beer)

D.1.1 Debugging Quesiton 1. For each bar Ben visits, find price of the most expensive and cheapest drink at that bar. Format the output as (bar, price), no duplicates.

The wrong query presented to the students are as follows:

```
WITH t1 AS (
  SELECT bar, price
  FROM serves
  WHERE price = (
      SELECT MAX(S1.price)
      FROM serves S1
      WHERE S1.bar = bar
    )
  UNION ALL
  SELECT bar, price
  FROM serves
  WHERE price = (
      SELECT MIN(S1.price)
      FROM serves S1
      WHERE S1.bar = bar
    )
)
SELECT t1.bar, t1.price
FROM t1, frequents
WHERE t1.bar = frequents.bar
  AND frequents.drinker = 'Ben';
```

The above query has two mistakes:

(1) `UNION ALL` creates duplicates when the most expensive and cheapest drinks share the same price.
(2) The `bar` in both scalar subqueries is referencing the wrong column. Without correct aliasing, both `bar` refer to the `bar` in `S1`, making the `WHERE` condition a tautology.

D.1.2 Debugging Quesiton 2. Suppose every time a drinker frequents a bar, he buys all his favorite beers at that bar. Find the expected weekly revenue of each bar and rank them by the revenue from high to low. The output should be in the format of (bar, revenue). If a bar is not frequented by any drinker, or it does not serve any beer, or none of its beer is liked by any drinker, output (bar, NULL).

The wrong query presented to the students is as follows:

```
SELECT S.bar,
  SUM(F.times_a_week) * SUM(S.price) AS revenue
FROM serves S,
  frequents F,
  likes L
WHERE S.bar = F.bar
  AND S.beer = L.beer
GROUP BY S.bar
ORDER BY revenue DESC;
```

The above query has three mistakes:

(1) The join predicate `F.drinker = L.drinker` is missing.
(2) The expression for the sum is incorrect as it will blow up the result. The correct expression is `SUM(F.times_a_week * S.price)`.
(3) There will be no "`NULL`" tuple produced by the query, i.e., bars which do not serve any beer / serve no beer liked by anyone will not be included in the result.

D.2 Brief Case Study for Debugging with LLM

To examine how large language models (LLMs) perform on query debugging tasks, we fed both debugging questions in the quiz to a LLM and verify the correctness of its response. We perform the test with Gemini-3-pro and GPT-5.4, which represented the state-of-the-art LLMs at the time this paper is written.

Since feeding the entire database instance to LLMs for debugging is usually not feasible, we used the following prompt for debugging and asked LLM to debug only based on the database schema and the query with the assumption that the query is known to be incorrect:

Prompt 1

```
You are a SQL expert who can debug semantically
incorrect query.

Consider the following database schema:
{{ Database Schema }}

Consider the following question:
{{ Question }}

The following query is proven to be semantically
incorrect:
{{ Query }}

What are the mistakes in the query?
```

To ensure consistency, for each debugging question, we fed the LLM with the same prompt simultaneously in five separate conversations. We obtained the following overall result:

- For debugging question 1: Gemini-3-pro caught both bugs in 4 conversations, and it missed bug (2) in the remaining conversation; GPT-5.4 caught both bugs in all 5 conversations.
- For debugging question 2, both models caught all bugs in all 5 conversations.
- No hallucination was spotted.

While the above results looked promising, we further test LLM's capability by removing the assumption that the query is wrong with the same setting in 5 isolated conversations:

Prompt 2

```
You are a SQL expert who can debug semantically
incorrect query.

Consider the following database schema:
{{ Database Schema }}

Consider the following question:
{{ Question }}

Is the following query semantically correct?
If not, what are the mistakes?
{{ Query }}
```

As a result, weaker assumption degrades the accuracy of the models:

- For debugging question 1: Gemini-3-pro caught both bugs in 3 conversations, and it missed bug (2) in one conversation and marked the query correct in one conversation; GPT-5.4 caught both bugs in 2 conversations and missed bug 2 in the remaining 3 conversations.
- For debugging question 2: Gemini-3-pro caught both bugs in all conversations while GPT-5.4 missed bug 3 in one conversation (it correctly identified the query would not produce `NULL` but gave the wrong reason).
- No hallucination was spotted.

D.2.1 Bar Raiser. We further challenged LLMs with a harder question. Given the following schema: Friends(uid1, uid2, start_ts, end_ts). Returns for each pair of users, maximal time periods during which they were friends.

The semantically incorrect query is the following:

```sql
WITH F(uid, start_ts, end_ts) AS (
  SELECT
    DISTINCT uid1,
    start_ts,
    COALESCE(
      end_ts, CURRENT_TIME + INTERVAL '1 day'
    )
  FROM
    Friends
),
Mystery(uid, start_ts, end_ts) AS (
  SELECT f1.uid, f1.start_ts, f2.end_ts
  FROM F f1, F f2
  WHERE
    f1.uid = f2.uid
    AND f1.start_ts < f2.end_ts
    AND NOT EXISTS (
      SELECT *
      FROM F fi
      WHERE
        fi.uid = f1.uid
        AND f1.start_ts < fi.end_ts
        AND fi.end_ts < f2.end_ts
        AND NOT EXISTS (
          SELECT *
          FROM F
          WHERE
            uid = fi.uid
            AND start_ts <= fi.end_ts
            AND fi.end_ts < end_ts
        )
    )
)
SELECT
  uid,
  start_ts,
  NULLIF(
    end_ts, CURRENT_TIME + INTERVAL '1 day'
  )
FROM Mystery m
WHERE
  NOT EXISTS (
    SELECT *
    FROM Mystery
    WHERE
      uid = m.uid
      AND start_ts <= m.start_ts
      AND m.end_ts <= end_ts
  );
```

In the above query, the `NOT EXISTS` condition in the outer `SELECT` block can never be satisifed, because both inequalities are non-strict (`<=`), any row sqlm will match itself (e.g., `m.start_ts <= m.start_ts` is always true). As a result, `EXISTS` is universally true, `NOT EXISTS` is universally false, and every single row eliminates itself.

For each model (i.e., Gemini-3-pro and GPT-5.4) and each prompt (Prompt 1 and 2 with different assumptions), we create 5 separate conversation and the results are the following:

- **Gemimi-3-pro:** With prompt 1, the bug in 4 conversations and missed it in one conversation; with prompt 2, it correctly identified the bugs in 3 conversations and missed it in 2 conversations.
- **GPT-5.4:** With prompt 1, it correctly identified the bug in 3 conversations but missed it in the other 2; with prompt 2, it only correctly identified it in one conversation.
- Both models hallucinated and mentioned other irrelevant bugs in almost all conversations.

On the other hand, this bug can be easy to spot with I-Rex: from the main query block, users can step into the execution of the `NOT EXISTS` subquery with any external reference row for `m.uid`, `m.start_ts`, `m.end_ts`, and quickly find out that the inequalities will always be evaluated to true as any external row will always "semi-join" with its itself in the inner `Mystery` table.

D.2.2 Summary. While Large Language Models (LLMs) are highly effective at debugging relatively simple SQL queries—particularly when operating under the strict assumption that a query is definitively flawed—their capabilities degrade significantly when faced with complex, multi-nested logic or when a user is merely uncertain about a query's correctness. Because LLMs act as probabilistic oracles rather than execution engines, they can easily hallucinate fixes or misinterpret the underlying data state. Consequently, there is a critical need for complementary tools to both ground the LLM's reasoning and allow users to rigorously verify the accuracy of its

proposed explanations. In both capacities, I-Rex provides a vital solution. By supplying a deterministic, step-by-step execution state, I-Rex serves as both an authoritative verifier for human users and a reliable, objective execution environment that future LLM agents can query to validate their own hypotheses.